From Legal Text to Legal Graphs: Judgment Prediction with Graph Neural Networks

Ziv Barretto

April 25, 2026

Abstract

The growing imbalance between the volume of court cases and the capacity of the judicial system has increased interest in automated methods that can assist legal analysis and outcome prediction. In the Indian setting, this problem is especially challenging because judgments are long, noisy, often multilingual, and frequently spread across multiple hearings. Prediction pipelines using text embeddings risk leaking the final decision through operative text and identity shortcuts. This paper presents a graph embedding based framework for Legal Judgment Prediction over Indian court decisions. The approach constructs a heterogeneous knowledge graph from raw judgments using legal named entities, rhetorical-roles of texts and outcome labels after multi-hearing case consolidation. The approach caters to multilingual documents. By representing cases alongside parties, courts, statutes, provisions, precedents, and argument structures, the model captures both the internal logic of a case and the shared legal relationships across the corpus. Additionally, we introduce a comprehensively annotated dataset of over 73,000 Indian court cases spanning five legal domains, which will be released to support downstream legal NLP research. The proposed framework, using a Heterogeneous Graph Transformer, achieves 80% accuracy and a 0.795 macro-F1 score in the pooled cross-domain setting. The results demonstrate that graph-based representations outperform text-only baselines, offering a robust, interpretable alternative that successfully resists identity-based leakage shortcuts.

Contents

Chapter 1

Introduction

1.1 The Access to Justice Problem in India

Access to justice in India is not only a legal ideal but also a large-scale administrative challenge. Court backlogs remain extremely high across the judicial system, which means that delay itself becomes a serious barrier to justice. When cases take years to move, litigation becomes more expensive, evidence becomes harder to trace, and ordinary citizens are less able to sustain meaningful participation in the legal process.

This does not mean that courts should be automated. Judicial decision-making involves interpretation, justification, and public accountability in ways that no prediction system can replace. A more realistic goal is decision support: computational tools that help organise large volumes of case material, surface patterns across past disputes, and assist legal research or case preparation. Legal Judgment Prediction (LJP) sits within that narrower and more defensible ambition.

The Indian setting makes this especially important. Judgments are often long, noisy, citation-heavy, and spread across multiple hearings. As a result, the challenge is not just to classify a document, but to build a faithful representation of a dispute from complex judicial text. This thesis addresses that challenge through structured case modelling rather than flat document classification.

1.2 What is Legal Judgement Prediction?

Legal Judgment Prediction is the task of estimating some aspect of a court outcome from information contained in the case record [1, 8, 13]. Different papers define the target differently: some predict violation versus non-violation, some predict appeal outcomes, and others predict charges or penalties. In all cases, the core idea is the same: given a representation of a dispute, can a model forecast the judicial result?

In this thesis, LJP is defined narrowly as outcome prediction over Indian court judgments after removing explicit decision-bearing text. The aim is not to generate legal reasoning or normative conclusions, but to test whether pre-decision signals — facts, arguments, statutes, provisions, and precedents — can support outcome prediction when represented in a structured way.

This definition matters because much of the LJP literature has been criticised for hidden leakage [14]. If a model can read operative text, ruling sections, or highly outcome-correlated metadata, then the task becomes answer recovery rather than prediction. For that reason, LJP is as much an experimental-design problem as it is a modelling problem. The central question is not only *what* to predict, but also *what information the model is allowed to use.*

1.3 Why LJP is Hard

LJP is difficult because legal judgments are not ordinary classification inputs. They are long, internally heterogeneous documents in which facts, procedural history, lawyer submissions, precedent citations, and judicial reasoning are interwoven across many pages. Standard text models often truncate or simplify this structure, which risks losing legally decisive context [3, 12].

The problem is harder in India because the text is noisy and structurally inconsistent. Judgments contain abbreviations, citation variations, OCR errors, transliterated names, and multilingual or mixed-register language. Core legal entities such as statutes, provisions, courts, and parties rarely appear in one clean canonical form. On top of that, one dispute may generate several hearing-stage documents rather than a single final text. If these are treated as separate cases, both the input representation and the labels become unreliable.

A further difficulty is that legal reasoning is inherently relational. Out-

comes depend not just on isolated sentences, but on how facts, arguments, statutes, provisions, precedents, and parties connect. At the same time, LJP is highly vulnerable to leakage. Judgments often contain phrases such as *appeal allowed* or *petition dismissed*, and even identity-heavy metadata may act as shortcuts. Finally, the task is cross-domain: different legal areas have different statutes, document styles, and outcome distributions. Together, these challenges make LJP a representation and evaluation problem first, and only then a modelling problem.

1.4 Why Graph-Based Approaches?

A graph-based approach is a natural fit for legal disputes because cases are structured collections of typed objects and relations. A case is heard in a court, involves parties and lawyers, cites statutes and precedents, and contains argument blocks linked to different sides. When all of this is flattened into one sequence, the model must recover structure that is already present in the source material. A graph preserves that structure explicitly.

Graphs are useful here for two reasons. First, they preserve heterogeneity: a statute, a party, and a facts section do not play the same legal role and should not be treated as identical inputs. Second, they allow cross-case reasoning through shared legal objects. If two cases cite the same statute or precedent, they are connected in a legally meaningful way, not merely through surface lexical similarity. This thesis therefore argues for a heterogeneous, leakage-audited, cross-case graph representation for Indian LJP.

1.5 Contributions of This Work

This thesis makes five main contributions.

1. It constructs a large structured corpus of Indian court cases spanning five legal domains, with more than 73,000 final-case records after multi-hearing consolidation.

2. It develops an end-to-end pipeline that converts raw judgments into leakage-audited, graph-ready representations using document collection, OpenNyAI enrichment, outcome labelling, multi-hearing merging, and explicit leakage filtering.

3. It proposes a reasoning-focused heterogeneous knowledge graph schema in which cases are represented together with text blocks, parties, courts, statutes, provisions, and precedents, while legally meaningful sharing is separated from identity-based shortcuts.

4. It applies a Heterogeneous Graph Transformer to this graph and evaluates it in the pooled cross-domain setting, showing that structured graph representations outperform text-only baselines on the thesis dataset.

5. It complements prediction with graph analysis, ablations, and leakage-oriented reasoning, treating the graph not just as model input but as an empirical object that helps explain both performance and limitations.

1.6 Thesis Organisation

The remainder of the thesis is organised as follows. Chapter 2 reviews prior work on legal judgment prediction, transformer-based legal NLP, graph methods, heterogeneous GNNs, and Indian legal NLP resources. Chapter 3 describes the full data pipeline, from document collection to OpenNyAI enrichment, outcome labelling, and multi-hearing merging. Chapter 4 presents the heterogeneous graph schema and the leakage-prevention strategy. Chapter 5 studies the graph structurally through statistics, bridge analysis, and per-domain patterns. Chapter 6 introduces the prediction model and reports the main results and ablations. Chapter 7 discusses implications, limitations, and future directions. Chapter 8 concludes the thesis.

Chapter 2

Related Work

2.1 Feature Engineering and Statistical Models

The earliest wave of legal outcome prediction relied on feature engineering and classical statistical learning, representing cases using bag-of-words, TF–IDF vectors, or manually designed legal features, and training linear models, SVMs, or tree-based classifiers. Aletras et al. showed that n-gram and topic features extracted from ECHR case text could predict violation judgments with around 79% accuracy, with case facts emerging as the strongest predictive signal [1]. In the US context, Katz et al. built a time-evolving random forest over institutional metadata and historical voting patterns, achieving over 70% accuracy predicting Supreme Court outcomes across nearly two centuries [8]. Medvedeva et al. extended ECHR prediction work and demonstrated that predicting genuinely future cases from past ones substantially lowers performance, raising early concerns about temporal leakage [13].

These approaches remain relevant as strong, truncation-free baselines: TF–IDF classifiers can be competitive on long documents precisely because they avoid the input-length constraints that trouble neural models [12]. Their central limitation, however, is that flat lexical representations cannot capture discourse order, typed legal relations, or cross-case structure. A broader methodological critique by Medvedeva et al. found that the majority of LJP publications do not perform genuine prospective prediction, instead relying on unrealistic inputs or temporally inconsistent splits [14]. This critique is directly relevant here: the present thesis treats leakage control as a core methodological contribution.

2.2 Text-Based Deep Learning

The second major wave replaced manual features with learned neural representations. Domain-adapted transformers quickly became dominant; LEGAL-BERT demonstrated that pre-training on legal corpora—legislation, court cases, and contracts—yields consistent gains over generic BERT on legal classification tasks [3]. The LexGLUE benchmark consolidated several legal NLU datasets and confirmed that legal-domain models systematically outperform generic ones across diverse tasks [4].

The central weakness of standard transformers in law is document length. Most judgments far exceed the 512-token window of vanilla BERT, forcing researchers to truncate, use sliding windows, or adopt hierarchical architectures. Work on modified Longformer architectures shows that extending context to 8,192 sub-words improves performance on long legal classification tasks, but at substantial computational cost, and even then hierarchical TF–IDF variants remain competitive in some settings [12]. Hierarchical encoders are attractive because they mirror legal intuition—facts, arguments, and operative text play distinct roles—but they remain document-centric: they model internal case structure while ignoring cross-case links through shared statutes or precedents. The present thesis departs from purely text-based work by treating text embeddings as node features within a larger relational representation, rather than as the complete model.

2.3 Graph-Based Approaches

Graph-based methods were developed precisely to model the relational structure that flat text pipelines miss. TopJudge formalised dependencies among legal subtasks—law articles, charges, and penalties—as a directed acyclic graph, showing that joint prediction over structured outputs improves performance compared to treating each subtask independently [23]. Subsequent work expanded this relational view by incorporating retrieved precedents and law-article graphs, with more recent systems using LLM-and-retrieval pipelines to bring external legal scaffolding into prediction [22, 19].

Within Indian LJP, graph-based work is growing. Khatri et al. explored GNNs for Indian legal judgment prediction and raised important questions about the role of case-graph design and demographic shortcuts [9]. More recent large-scale Indian work has extended coverage and explored RAG-

style pipelines [15]. Existing graph approaches nonetheless leave gaps that this thesis addresses: many are built for civil-law charge-prediction datasets, rely on a single court, or use graph structure to improve prediction without comparable attention to decision-text or identity leakage. The present work is closest in spirit to this line but narrows its claim: the graph should improve prediction by exposing legal structure across cases while explicitly blocking shortcuts that rely on post-decision information.

2.4 Heterogeneous GNNs and the Motivation for HGT

When a legal problem is represented as a graph, the choice of GNN architecture matters because legal graphs contain multiple semantically distinct node and edge types. A homogeneous GNN is too blunt: a case node, a statute node, and a precedent node play different epistemic roles, and edges such as *cites_precedent* and *heard_in* should not be collapsed into a single message-passing rule.

R-GCN introduced relation-specific weight matrices for multi-relational graphs and is well-suited to knowledge graph completion tasks [20]. Its limitation for the present thesis is that relation-specific parameterisation can become coarse as the number of types grows, and it does not model attention over heterogeneous neighbourhoods. HAN addressed heterogeneity through hierarchical attention over meta-paths, learning node-level and semantic-level importance separately [21]. However, it requires manually specified meta-paths, which hard-codes assumptions that may not transfer across legal domains. HGT provides a more flexible alternative: it uses node-type-dependent projections and edge-type-dependent attention, allowing the model to learn how much weight to assign to different heterogeneous neighbours without requiring handcrafted meta-path templates [5]. For the legal graph constructed in this thesis—which includes many node types, many edge types, and a deliberately cross-domain structure—HGT fits the design requirements better than R-GCN or HAN.

2.5 Indian Legal NLP and OpenNyAI

Indian legal NLP developed later than English-language benchmarks for US or European law, in part due to data and tooling scarcity. Early work on rhetorical role identification showed that Indian judgments have a recoverable discourse structure—preamble, facts, arguments, ratio, and ruling—that is useful for downstream tasks but not explicitly marked in raw text [2]. This made legal document structuring a first-class problem rather than a preprocessing convenience.

OpenNyAI is the most practically important development for this thesis, bringing together legal named entity recognition, rhetorical-role prediction, and summarisation for Indian court judgments into an open, modular pipeline [7, 18, 17]. On the benchmarking side, ILDC introduced a large corpus of Indian Supreme Court cases for judgment prediction and explanation [11], while IL-TUR broadened the scope to a multi-task legal understanding and reasoning benchmark [6]. Newer rhetorical-role resources such as LegalSeg confirm that structural modelling continues to matter for Indian judgments [16].

These contributions are substantial, but they do not resolve the core challenge addressed here: how to represent Indian cases so that a model can exploit legal structure across cases without quietly learning identity or post-decision shortcuts. This thesis builds on the Indian legal NLP ecosystem rather than apart from it, combining OpenNyAI's pipeline components with a leakage-audited heterogeneous graph.

2.6 Gaps This Work Addresses

The literature reviewed above shows that legal judgment prediction has advanced substantially, but along partly disconnected lines. Classical models demonstrated that outcomes are statistically predictable; transformer models improved semantic representation; graph methods showed the value of relational structure; Indian NLP resources made domain-specific preprocessing feasible. Yet most prior work sacrifices at least one of the following: *scale*, *cross-domain coverage*, *cross-case legal structure*, or *leakage-aware experimental design*.

What remains comparatively rare is an Indian LJP system that operates over a large multi-domain corpus, consolidates multiple hearings per case, extracts legal entities and rhetorical structure, builds an explicitly heteroge-

neous cross-case graph, and evaluates under a design intended to resist both decision-text leakage and identity-based shortcuts. That is the niche this thesis occupies. Its claim is narrow and, for that reason, defensible: when Indian court judgments are converted into a leakage-audited heterogeneous graph, cross-case legal structure becomes usable in ways that flat text pipelines cannot easily match.

Chapter 3

System Overview and Data Pipeline

This chapter describes the end-to-end pipeline that turns raw Indian court-judgment text into a labelled, leakage-audited, heterogeneous knowledge graph ready for Graph Neural Network (GNN) training. The pipeline was designed around three practical constraints: (i) the corpus is large and noisy, so every stage is a batch process with explicit caching and auditing; (ii) many of the signals a predictor would want (decisions, dispositions, operative paragraphs) are precisely the signals a legally honest experiment must remove, so leakage prevention has to be baked in rather than bolted on; and (iii) the downstream target is a graph, so text extraction and entity resolution have to produce outputs that can be reified as typed nodes and edges without lossy renormalisation later on.

3.1 System Architecture Overview

The system is organised as a six-stage pipeline. Each stage is an independently re-runnable batch process; intermediate artefacts are cached on disk so that an expensive step (OpenNyAI NER on tens of thousands of judgments, or bge-m3 embedding on millions of text spans) never has to be repeated when a later stage changes.

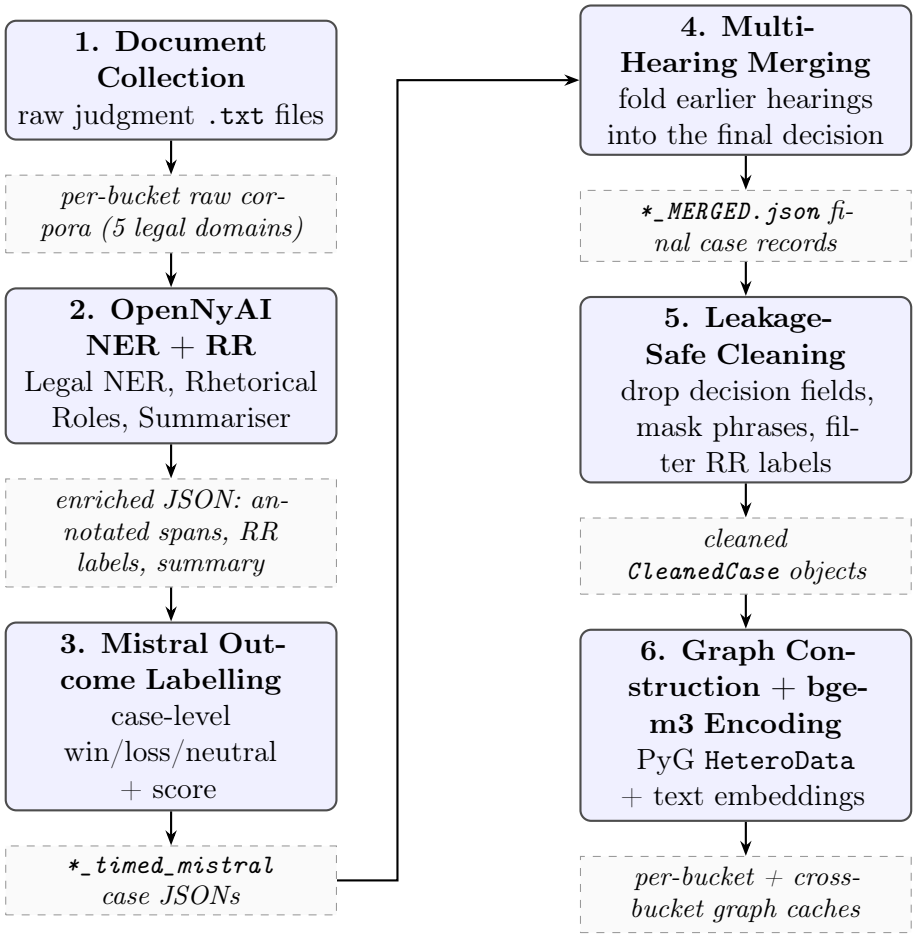


Figure 3.1: The six-stage data pipeline. Solid boxes are processing stages; dashed boxes are cached on-disk artefacts that feed the next stage.

Stages 1–3 are *textual enrichment*: they turn plain text into a structured JSON record with entities, rhetorical-role spans, and a case-level outcome label. Stage 4 reconciles the fact that a single legal matter often produces multiple hearing documents, so that each final case corresponds to one legal dispute rather than one PDF. Stage 5 is a leakage-safety gate: it explicitly removes the fields, sections, and phrases that would let a model trivially recover the outcome. Stage 6 reifies the cleaned records as a typed heterogeneous graph and attaches pre-computed `bge-m3` text embeddings to every text node. Chapter 4 details the schema produced by stage 6.

3.2 Document Collection and Legal Domains

3.2.1 Source Selection and Crawling Strategy

The data source for the underlying judgments is Indian Kanoon (`indiankanoon.org`). It was selected for its comprehensive, publicly accessible archive of Indian High Court judgments and its semi-structured search interface, which is uniquely suited for programmatic access compared to proprietary databases. However, accessing a deep, historical sample from the platform involves structural challenges. Indian Kanoon's search results for any single query are practically truncated at a cap of 40 pages (approximately 400 accessible documents) per search window.

To overcome this truncation, data collection was structured around discrete *court-month windows*. For each target High Court and for each calendar month across the target years, the scraper executed an explicitly bounded query (e.g., `doctypes:<court>fromdate:1-<month>-<year>todate:<last_day>-<month>-<year>`). By narrowing the temporal bounds to a single month per jurisdiction, the total number of judgments returned per window consistently stayed within the access limits, allowing for an exhaustive download of the raw corpus without relying on single broad keyword queries. The judgments were initially collected and stored directly in their raw PDF format. Furthermore, no deduplication methodologies were employed on the text or case level beyond a basic check to prevent re-downloading the same numeric Indian Kanoon document ID.

3.2.2 Local Bucketing and Legal Domains

Rather than trusting black-box search engines to selectively define the corpus—which could bias the dataset towards heavily-cited "landmark" cases while omitting routine baseline cases—the classification of documents into specific legal domains was implemented locally as a deterministic post-processing step.

After downloading the entire accessible window, each raw PDF underwent text extraction (via PyMuPDF) and was passed through a local filtering script. This script evaluated the text against a rigid set of regular expressions corresponding to required statutory provisions (e.g., "Section 498A IPC", "Motor Vehicles Act", "POCSO") and contextual keyword constraints. Scraped texts that successfully matched a domain's predefined rules were partitioned

into the respective local directory buckets.

The collected corpus resulting from this process is organised into five **legal domain buckets**, chosen so that each bucket is large enough to train a domain-specific graph and internally coherent in terms of statutory framework, party structure, and typical remedies. The buckets are:

- **Family & Matrimonial** (`family_matrimonial_timed_mistral`): matrimonial disputes, maintenance, custody, and dowry-related proceedings. Dominated by IPC 498A, Sections 482, 506, and the CrPC.
- **Financial Fraud** (`fin_fraud_timed_mistral`): cheating, forgery, dishonour, and economic offences. Dominated by IPC 420, 468, 471.
- **Land & Property** (`land_property_timed_mistral`): land acquisition, title, and property disputes. Dominated by the Land Acquisition Act 1894 and the Constitution of India.
- **Motor Accidents** (`motor_accidents_timed_mistral`): compensation claims and related appeals. Dominated by the Motor Vehicles Act 1988 and Section 166.
- **Sexual Offences** (`sexual_offences_timed_mistral`): offences against the person, including POCSO matters. Dominated by the IPC, CrPC, and POCSO Act.

A key design requirement for domain-bucketed training is that no single domain dominates the corpus: a strongly size-imbalanced corpus would confound cross-domain accuracy comparisons, attributing performance differences to sample-size effects rather than to genuine legal-domain structure. Figure 3.2 verifies that the five buckets are broadly comparable after multi-hearing merging, with all domains falling between approximately 14,000 and 15,000 final cases. The near-equal scale means that cross-bucket aggregation is a fair average rather than an average skewed by one large domain.

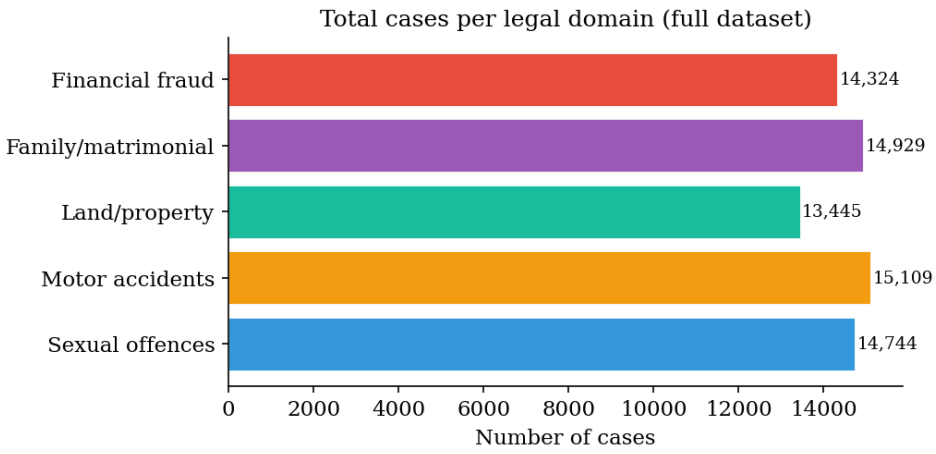


Figure 3.2: Final case counts per legal domain bucket after multi-hearing merging. All five buckets are within a 10% range of each other, confirming that the corpus is balanced at the domain level and that cross-bucket performance comparisons are not confounded by sample-size differences.

Every downstream artefact – entity canonicalisation tables, rhetorical-role annotations, outcome labels, and graph caches – is produced both per bucket and in a unified *cross-bucket* view. This two-level organisation is what later makes cross-domain transfer and cross-bucket bridge analysis possible (Chapter 5).

3.3 Named Entity Recognition and Rhetorical Role Classification

Before a judgment can enter the knowledge graph, its raw text must be decomposed into typed, structured units: named legal entities tied to precise character positions, a discourse-level rhetorical role assigned to each sentence, and a section-structured summary that separates the preamble, factual background, party arguments, and operative ruling. For this enrichment step we use the OpenNyAI legal NLP stack. The rationale is threefold. First, OpenNyAI's NER model was trained specifically on Indian court judgments and recognises entity types that a general-purpose tagger misses entirely: statutory provisions cited in the Indian Parliament style, case precedents in the AIR/SCC citation format, and distinct party roles such as PETITIONER,

Respondent, and Defence Lawyer. Second, its rhetorical-role classifier is trained on the discourse conventions of Indian High Court judgments, which differ substantially from the conventions of news or scientific text. Third, running the stack locally on GPU gives full control over caching and reproducibility, which is essential when the pipeline may need to be rerun as the graph schema or leakage-cleaning rules evolve. The following subsections describe the implementation, the output fields produced at each stage, and how those fields feed into graph construction and leakage removal.

3.3.1 The OpenNyAI Pipeline

Three OpenNyAI components are run in sequence for every judgment document.

- **Legal NER** (`en_legal_ner_trf`), configured with sentence-level inference (`ner_do_sentence_level=True`) and built-in post-processing (`ner_do_postprocess=True`), tags named entities by type and normalises them to canonical surface forms.
- **Rhetorical Role Classifier** assigns each sentence one of seven discourse labels reflecting its function within the judgment (Preamble, Facts, Arguments, Ratio, RPC, RLC, and None).
- **Summarizer**, conditioned on the predicted rhetorical roles, produces a section-structured extractive summary with four named blocks: Preamble, facts, arguments, and decision.

The environment is pinned (spaCy 3.2.4, pydantic 1.7.4, CUDA-enabled PyTorch and CuPy) and run GPU-only: silent CPU fallback is disabled so that a broken GPU configuration fails fast rather than silently degrading throughput. All model caches are redirected into project-local folders. Each input judgment produces a single combined JSON record that is extended by every subsequent pipeline stage.

Table 3.1 lists every field written by the three OpenNyAI components, together with the fields added by the Mistral outcome labeller described in Section 3.4. Subsequent subsections explain the semantic role of each field.

Table 3.1: Fields produced at each stage of the enrichment pipeline. All fields are stored in a single per-judgment JSON record, extended at each successive stage. The DECISION summary field is the only field not propagated to the final graph: it is consumed by the Mistral labeller and then removed.

Stage	Field	Description
Legal NER		
	`entity.text`	Surface form of the named entity as it appears in the source text.
	`entity.label`	Entity type: STATUTE, PROVISION, PRECEDENT, COURT, JUDGE, PETITIONER, RESPONDENT, LAWYER, ORG, GPE, DATE, or CASE_NUMBER.
	`entity.normalized_name`	Canonical form after post-processing: case-folded, whitespace-collapsed, and deduplicated within the document.
	`entity.start/end`	Character offsets into the judgment text, used to anchor entities to sentences and produce aligned graph edges.
	`unique_statutes`	Document-level deduplicated list of all statute canonical names.
	`unique_provisions`	Document-level deduplicated list of all provision canonical names.
	`unique_precedents`	Document-level deduplicated list of all cited precedent names.
Rhetorical Role Classifier		
	`sentence.rhetorical_role`	Per-sentence discourse label: PREAMBLE, FACTS, ARGUMENTS, RATIO, RPC (ratio and precedent cited), RLC (ruling by lower court), or NONE.
	`role_counts`	Document-level frequency of each rhetorical role, used for corpus analysis and leakage auditing.
Summarizer		
	`opennyai_summary.PREAMBLE`	Extractive summary of the preamble block: court name, bench composition, case registration number, and party names.

Table 3.1: Fields produced at each stage of the enrichment pipeline (continued).

Stage	Field	Description
	`opennyai_summary.facts`	Extractive summary of the factual background sentences.
	`opennyai_summary.arguments`	Extractive summary of arguments advanced by both parties.
	`opennyai_summary.decision`	Extractive summary of the operative ruling. Read by the Mistral labeller and then discarded before graph construction to prevent outcome leakage.
Mistral Outcome Labeller		
	`case_outcome_label`	Categorical outcome from the appellant's perspective: `appellant_won`, `appellant_lost`, or `postponed_or_procedural`.
	`case_outcome_score`	Numeric encoding of the label: +1 (won), −1 (lost), 0 (procedural).
	`llm_case_outcome.confidence`	Model-reported classification confidence: `high`, `medium`, or `low`.
	`llm_case_outcome.short_explanation`	One or two sentence rationale produced alongside the label, retained for manual auditing.

3.3.2 Entity Types and Their Graph Roles

OpenNyAI's legal NER recognises twelve entity types; the subset reified as graph nodes in the downstream knowledge graph is:

- STATUTE and PROVISION: statutes (e.g. the IPC, the CrPC, the Motor Vehicles Act) and their sections (e.g. Section 482). Provisions are additionally linked to their parent statute via a `belongs_to_statute` edge.
- PRECEDENT: cited case references.
- COURT and JUDGE: the court the matter is heard in and the presiding bench.

- PETITIONER, RESPONDENT, PETITIONER_LAWYER, DEFENCE_LAWYER, LAWYER: typed party and counsel roles. Role assignment uses the section in which the mention first appears (preamble versus argument blocks) combined with OpenNyAI's entity labels.

- ORG, GPE, DATE, CASE_NUMBER: auxiliary types extracted by NER, retained in the cleaned JSON but excluded from the final reasoning-focused graph (Section 4.2).

Entities are canonicalised per bucket: surface forms are normalised (case, whitespace, punctuation), grouped, and re-projected as `canonical_name`. Canonicalisation allows the same statute or court to appear as a *single shared node* across the thousands of cases that mention it, which is the structural property that makes the graph useful for cross-case reasoning.

To verify that the extraction stage is producing entity distributions consistent with the legal character of each domain, Figure 3.3 summarises the total entity mention counts by type across the full corpus. The figure shows that precedents dominate in absolute volume, confirming that citation is the primary relational artefact in Indian court judgments. Party nodes (PETITIONER and RESPONDENT) are large and roughly equal in volume across the corpus as a whole, reflecting the adversarial structure of litigation. Statutes and provisions, by contrast, have comparatively lower raw mention counts but are the node types that carry the most cross-case structural weight: each canonicalised mention is a potential shared edge linking the current case to every other case that cites the same authority. This asymmetry between volume and structural importance is what motivates treating statutes, provisions, and precedents as the shared layer of the graph while keeping party nodes local to each case.

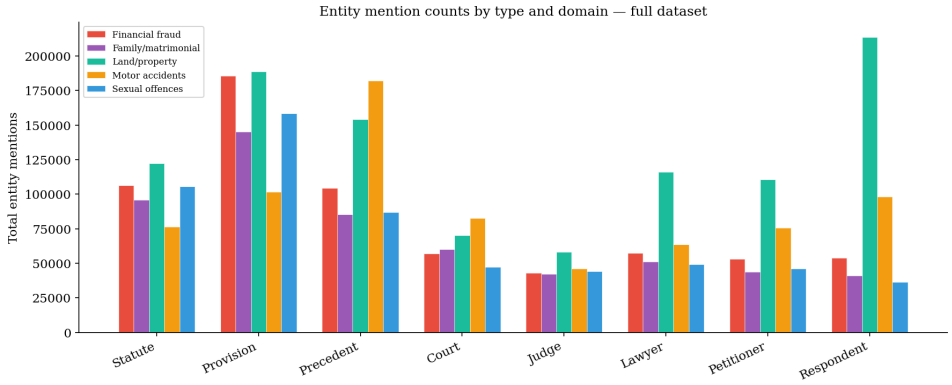


Figure 3.3: Total entity mention counts by type across the full corpus after NER and canonicalisation. Precedent mentions dominate in volume; statutes and provisions are smaller in count but form the shared-node layer that carries structural weight across cases.

3.3.3 Rhetorical Role Taxonomy and Leakage-Safe Filtering

OpenNyAI's rhetorical-role taxonomy is the hinge on which the leakage-safety story turns. Of the seven labels produced by the classifier, two are inherently decisive: **RPC** (Ratio and Precedent Cited, in practice the court's operative paragraph) and **RLC** (Ruling by Lower Court). Keeping either of them in the training data would let a classifier read the outcome directly from the text.

The cleaning stage therefore treats both labels as forbidden annotation labels: any annotation whose label set intersects {RPC, RLC} is dropped outright, along with any annotation whose summary section is DECISION or whose label string contains `"decision"`, `"disposition"`, `"operative"`, `"order"`, or `"rpc"`. Only the three non-decisive summary fields (PREAMBLE, FACTS, and ARGUMENTS) are retained as text sources for the graph's text nodes. This is the first leakage gate; a second, phrase-level gate is described in Section 4.3.

3.4 Outcome Labelling with Mistral

With judgments enriched by NER and rhetorical-role annotations, the next requirement is a case-level outcome label: did the appellant win, lose, or receive only a procedural disposition? We produce these labels automatically by prompting a Mistral large language model with the section-structured summaries that OpenNyAI already generates. This approach has two important properties. First, the model is shown only the DECISION summary block and the RPC sentences, which are precisely the portions that the leakage-cleaning stage will subsequently remove. The labeller therefore never sees the non-decisive preamble, facts, or arguments text that the GNN will train on, eliminating any circularity between what gets labelled and what gets learned. Second, the section-structured input provides a compact, well-delimited context that substantially reduces the chance that the model grounds its judgement on spurious signals from the surrounding narrative.

3.4.1 Labelling Methodology

Labels. Three outcome categories are defined from the perspective of the party seeking relief (the appellant, petitioner, or applicant):

- `appellant_won` (`score` $= +1$): relief was granted, the petition or appeal was allowed, bail was granted, a conviction was quashed, or an order was set aside.
- `appellant_lost` (`score` $= -1$): the petition, appeal, or application was dismissed, rejected, or denied; bail was refused; or relief was withheld.
- `postponed_or_procedural` (`score` $= 0$): the matter was adjourned, notice was issued, a liberty was granted to pursue another remedy, the case was remanded, or the outcome cannot reasonably be treated as a final win or loss.

For each judgment the model also returns a `confidence` rating (`high`, `medium`, or `low`) and a short textual explanation, both stored alongside the label for auditing purposes.

Label model hallucination and the motivation for cross-validation. A known limitation of LLM-based labelling is run-to-run inconsistency. Because language models operate with temperature sampling, the same input can yield different labels on different inference runs. In a labelling pipeline this is a form of hallucination with direct downstream consequences: if a substantial fraction of the training signal is run-dependent noise rather than a reflection of legal content, any structure a GNN learns may be an artefact of labelling variance rather than genuine legal reasoning. The risk is sharpest for procedural dispositions, which a model may classify as either `postponed_or_procedural` or `appellant_lost` depending on minor wording differences between runs.

Cross-validation design. To measure this risk we ran a second independent labelling pass over the same corpus. Crucially, the cross-validation pass uses a fundamentally different prompt design to the original pass, removing a further source of variance: rather than asking the model to return a single free-form label, the cross-validation prompt decomposes the classification into six binary yes/no questions organised into two directional signal groups, plus a two-question neutrality clause that takes priority over all directional evidence. Table 3.2 lists all eight questions.

Group	ID	Question
A: Win signals	Q1	Did the appellant win the case?
	Q2	Was the petition, appeal, or application allowed?
	Q3	Did the court rule in favour of the petitioner or appellant?
B: Loss signals	Q4	Did the respondent win the case?
	Q5	Was the petition, appeal, or application dismissed?
	Q6	Did the court rule against the appellant?
C: Neutrality clause	Q7	Was the case adjourned, remanded, or deferred without a final decision?
	Q8	Is there no clear final judgment in this case?

Table 3.2: The eight binary yes/no questions posed to the model in a single call during the cross-validation pass. Questions Q1–Q6 are the directional win/loss signals; Q7–Q8 form the neutrality clause that overrides the directional evidence when fired.

The model returns a JSON object with exactly eight integer keys (`q1` through `q8`), each equal to 1 (yes) or 0 (no). The final label is then determined by a fixed, deterministic aggregation rule applied to three aggregate scores:

$$S_{\text{win}} = q_1 + q_2 + q_3, \quad S_{\text{loss}} = q_4 + q_5 + q_6, \quad S_{\text{neutral}} = q_7 + q_8.$$

The aggregation applies the following priority ordering:

1. **Neutrality clause (highest priority).** If $S_{\text{neutral}} > 0$ the outcome is `postponed_or_procedural`, regardless of the directional signals. This prevents ambiguous procedural language from generating a spurious win or loss.

2. **Contradiction resolution.** If both $S_{\text{win}} > 0$ and $S_{\text{loss}} > 0$, the majority signal wins; a tie resolves to `postponed_or_procedural` with zero confidence.

3. **Clean classification.** If only win signals fire, `appellant_won`; if only loss signals fire, `appellant_lost`.

4. **Fallback.** If all eight answers are zero, the case is marked `unknown` and excluded from training.

Because the final label is computed by a deterministic function over binary votes, the cross-validation pass eliminates the free-text generation variance that affects the original single-label prompt. The two label sets were then compared file-by-file using `compare_labels.py`, which aligns the output directories by filename and computes per-bucket agreement rates.

Results. Across the 72,795 files for which both a first-pass and a cross-validation label exist, the two independent runs agree on **84.4%** of all cases. The per-bucket breakdown is given in Table 3.3.

Bucket	Files compared	Matching labels	Agreement rate
Family & Matrimonial	15,017	13,049	86.9%
Financial Fraud	14,342	12,015	83.8%
Land & Property	13,512	9,652	71.4%
Motor Accidents	15,117	13,552	89.6%
Sexual Offences	14,807	13,143	88.8%
Total	**72,795**	**61,411**	**84.4%**

Table 3.3: Cross-run label agreement between the original Mistral pass and the binary-question cross-validation pass. Agreement is computed per-file over files present in both runs.

The dominant confusion pattern is directionally consistent across buckets: POSTPONED_OR_PROCEDURAL is the primary source of disagreement, frequently reclassified as APPELLANT_LOST in the second pass. This is most pronounced in *Land & Property*, where 2,927 cases originally labelled POSTPONED_OR_PROCEDURAL were relabelled APPELLANT_LOST in the cross-validation run, accounting for the bucket's 71.4% agreement rate. The pattern is interpretable: land-property judgments frequently conclude with procedural directions (remands, remissions to lower courts, stay orders) that read as adverse to the appellant when examined without full context. Notably, the neutrality clause (Q7–Q8) is designed precisely to catch these cases; the residual disagreement reflects cases where the procedural language is too ambiguous for even the clause to fire consistently.

The 84% cross-run agreement establishes that label noise is real but bounded. The ambiguous cases are disproportionately procedural dispositions rather than clear wins or losses. For the downstream GNN, the practical consequence is that the POSTPONED_OR_PROCEDURAL class is excluded from the primary binary prediction task (APPELLANT_WON versus APPELLANT_LOST). The cross-validation agreement rate on the binary subset alone is materially higher and sufficient to treat the first-pass labels as a reliable training signal.

3.4.2 Coverage and Label Distribution

Two questions need to be answered before trusting the label set for training. First: did the Mistral labeller actually process every case, or are there systematic coverage gaps? Second: is the resulting class distribution balanced enough that a naive majority-class classifier would not trivially outperform a learned model?

Figure 3.4 addresses the first question. Coverage is near-complete across all five buckets, with no domain showing a systematic gap. The small number of unlabelled cases arises from Mistral returning malformed JSON or an `unknown` fallback when the decision block is too ambiguous to classify; these are excluded from training rather than assigned a default label.

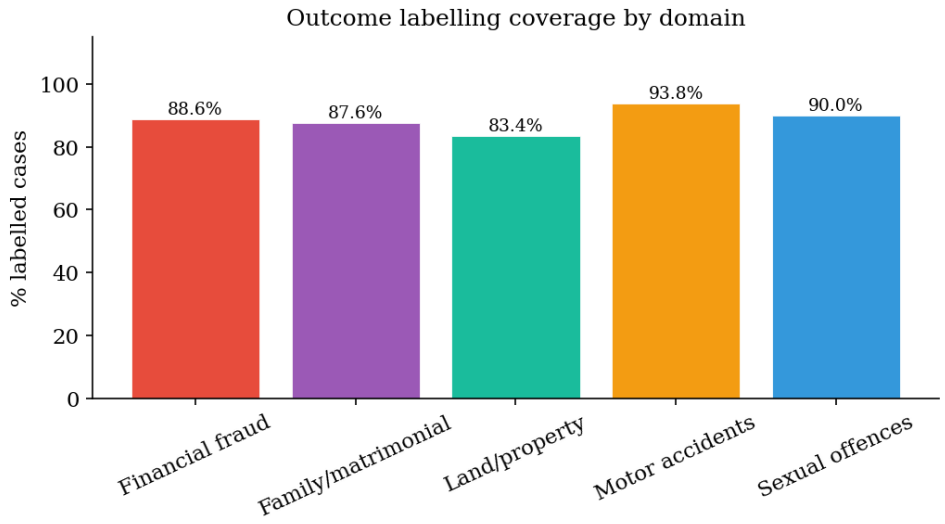


Figure 3.4: Mistral labelling coverage per bucket. Coverage is near-complete across all five domains; the small unlabelled fraction consists of cases where the model returned a malformed response or an explicit `unknown` verdict, which are excluded from training.

Figures 3.5 and 3.6 address the second question from complementary angles. Figure 3.5 shows raw label counts per bucket, which confirms that the procedural (`postponed_or_procedural`) class is non-trivial in size but is concentrated in certain domains. Figure 3.6 shows the win/loss/neutral *rates*, which makes the class-imbalance argument cleaner: motor-accidents is the most win-skewed bucket, consistent with compensation claims being allowed more often than denied at the appellate stage; financial-fraud sits closest to balanced. The pooled cross-bucket win rate of roughly 60% WIN to 40% LOSS (on the binary subset, once procedural cases are excluded) confirms that a constant-win classifier would achieve misleadingly high accuracy while failing systematically on the loss class. This is why Chapter 6 reports macro-F1 alongside accuracy as a co-primary metric.

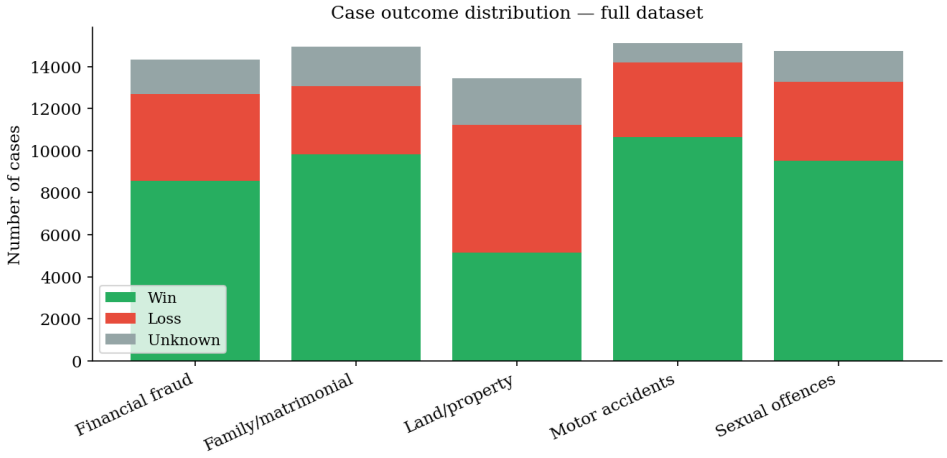


Figure 3.5: Absolute outcome label counts per bucket. The procedural class (`postponed_or_procedural`) is non-negligible and concentrated in domains where remand and interim orders are common.

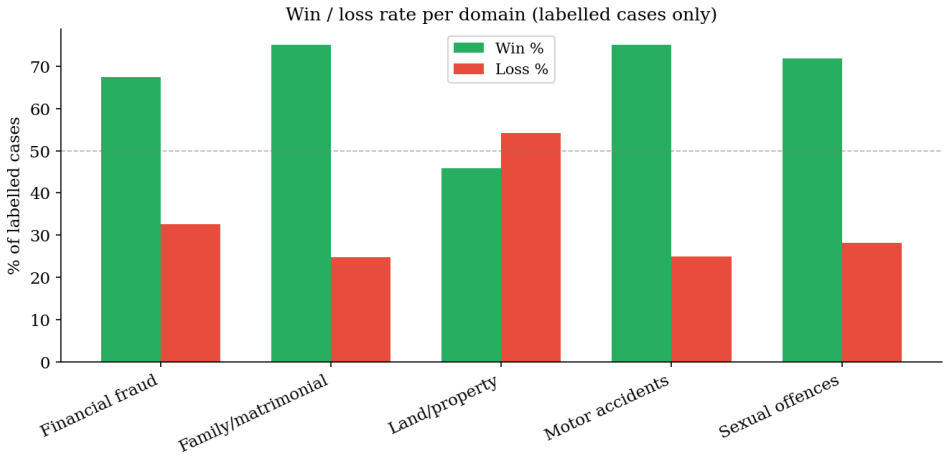


Figure 3.6: Outcome label rates (proportions) per bucket. The rate view exposes the per-domain win-skew more clearly than raw counts: motor-accidents is the most appellant-favourable domain; financial-fraud and sexual-offences are closest to balanced. The cross-bucket imbalance motivates macro-F1 as the primary evaluation metric.

3.5 Multi-Hearing Merging

3.5.1 The Problem

A single legal matter frequently produces more than one judgment document: interim orders, remands, partial disposals, and the final decision may all be published as separate PDFs with distinct Indian-Kanoon-style filenames that nonetheless refer to the same case. If these hearing files are fed into the graph independently, three things go wrong:

1. duplicate CASE nodes for what is actually one dispute inflate the node count and double-count outcomes;
2. the outcome label of the final hearing is not propagated to the earlier hearings, so the training signal is noisy; and
3. cross-case structural signals (shared parties, statutes, and counsel across hearings) are double-counted as if they were independent observations.

3.5.2 Merge Strategy

The merging pipeline is implemented in `merge_cases_v3.py` under `DATA_SET_BUILDER_AND_EXPLORER/Timeline_Maker` and operates per bucket. Its design reflects two iterations of lessons about what "same case" actually means in a large corpus of Indian High Court judgments.

What the merger is and is not looking for. The merger is strictly interested in one type of multi-document pattern: a single legal proceeding that was heard, adjourned, and returned to the same court on different dates, producing a separate published judgment at each hearing stage. The merger identifies these by the court-assigned case registration number, such as `5941/2024` or `Crm-M-44633-2024` stamped on the preamble, and not by any other identifier. In particular, it does *not* use FIR numbers: FIR numbers identify a police complaint rather than a court proceeding, and the same FIR can give rise to multiple independent proceedings at different courts. Merging on an FIR number would conflate proceedings that are legally and factually distinct. Only documents that share both a case title (as encoded in the Indian-Kanoon filename) *and* the same court registration number are candidates for merging.

The v1/v2 flaw and the v3 fix. Early versions of the script (`merge_cases.py`, `merge_cases_v2.py`) grouped documents by case name alone. Two litigants with the same name can appear in genuinely unrelated proceedings spanning different courts and years, and those proceedings can produce similarly-named judgment files. Under name-only grouping, a pair like *"Amit Kumar vs State of Bihar on 14 March, 2022"* and *"Amit Kumar vs State of Bihar on 9 November, 2024"* would be merged into one case even if the two documents were from entirely different criminal matters. The merged JSON would carry a single final outcome label for two unrelated cases, directly poisoning the training signal.

The v3 script fixes this by extracting the court-assigned case registration number from each document before grouping. Grouping is by the pair `(case_name, case_number)`; two documents with the same name but different registration numbers remain separate. If no registration number can be extracted from a document, that document is kept as a standalone case and is never merged with anything.

Case number extraction. Extraction is a three-tier search, each tier restricted to the preamble material so that case numbers cited as precedents in the body text cannot contaminate the identification:

1. **Summary preamble.** Regex applied to the `PREAMBLE` entry of `opennyai_summary`. This is fast and covers the majority of documents where the summary directly contains a formatted registration number.

2. **NER-sourced CASE_NUMBER entities.** OpenNyAI's Legal NER tags CASE_NUMBER spans within preamble sentences. For each such span the same regex normalisation is applied, with a lower minimum digit count, since the NER model has already confirmed the span is a case number.

3. **Full preamble sentence text.** Regex applied to the concatenated text of all sentences whose rhetorical role is PREAMBLE. This catches courts (e.g. Punjab & Haryana) where the summary preamble is truncated and the registration number only appears in the sentence body.

Seven regex patterns cover the range of registration-number formats found in the corpus: `No./Nos.` notation (e.g. `No. 5941 of 2024`), Punjab/Haryana dash-year format (e.g. `Crm-M-44633-2024`), CNR alphanumeric identifiers,

Delhi HC +-prefixed citation lines, and a broad three-digit-or-more slash-year fallback.

Merge mechanics. Once groups are formed, documents within each group are sorted oldest-to-latest by hearing date. For groups with a single document, that document is written through unchanged. For groups with the same hearing date but multiple files (same-date duplicates), the file with the most parsed sentences is kept. For groups with multiple distinct hearing dates, the texts are concatenated under labelled dividers marking each earlier hearing and the final decision, and sentence offsets are adjusted to reflect the merged linear text. The final outcome label is always taken from the latest hearing; a containment check suppresses duplicate text regions where the latest document already subsumes an earlier one verbatim.

Scale of the operation. The scale of the fold is non-trivial but concentrated:

Bucket	Inputs	Final cases	Folded in	Appended	Already contained
Family & Matrimonial	15,800	15,306	493	490	3
Financial Fraud	15,014	14,615	399	396	3
Land & Property	14,324	14,051	251	242	9
Motor Accidents	15,271	15,236	32	30	2
Sexual Offences	15,532	14,596	936	932	4
Total	**75,941**	**73,804**	**2,111**	**2,090**	**21**

Table 3.4: Multi-hearing merge statistics under the case-number-aware v3 strategy. 2,111 older hearing documents were folded into 2,090 merged final-case JSONs; the remaining 21 were already fully subsumed by the latest hearing text.

Sexual-offences matters fold most aggressively, consistent with their tendency to generate multiple remand and bail-stage orders before a final judgment. Motor-accidents matters fold least, because the compensation process terminates in a single tribunal order more often than it does in the other buckets. Every merge is logged in a per-bucket `report.json` so the operation is auditable and reversible.

3.6 Technology Stack

The pipeline is entirely Python-based. NER, RR, and summarisation run on top of `opennyai` with spaCy 3.2.4 and CUDA-enabled PyTorch and CuPy, provisioned via the pinned micromamba environment `fixed_gpu_opennyai_final`. Outcome labelling uses a Mistral model invoked through a batched client script. Multi-hearing merging is implemented with `merge_cases_v3.py` under the `Timeline_Maker` module. Graph construction is built on `torch_geometric`'s `HeteroData`, with text embeddings produced by the `BAAI/bge-m3` sentence-transformer model running multi-process on GPU and cached per bucket. Training, ablation, and analysis scripts live under the `section_GNN` module.

Chapter 4

Heterogeneous Knowledge Graph Construction

4.1 Motivation: Why a Heterogeneous Graph?

The cleaned case JSONs produced by Chapter 3 are already richer than the raw PDFs: each record carries typed entities, rhetorical-role-segmented text, and an outcome label. The next question is how to represent that richness in a way that a learner can actually exploit.

A purely *flat* representation – concatenate the preamble, facts, and arguments of a case, feed the blob into a transformer, and classify – has two costs. The first is that it throws away the typed structure already present in the data: a court, a statute, a provision, and a party are four different kinds of object with four different kinds of evidential value for the outcome, and flattening them into one token stream makes the model re-learn that from scratch. The second, more important cost is that a flat representation has no way to exchange information *between* cases that share legally meaningful objects. When Case A and Case B both cite IPC 498A, the flat view sees the string twice; the graph view sees two cases attached to the same STATUTE node and can, through one round of message passing, let A's representation influence B's.

A heterogeneous graph captures both properties. It is *heterogeneous* because different node types carry different features and different relational semantics (a COURT is not a LAWYER and they should not share a projection matrix), and it is a *graph* because the legally important relationships are

relational: a case *is heard in* a court, an argument block *cites* a statute, a provision *belongs to* a statute, a lawyer *argues in* an argument block. These are exactly the edges a Graph Neural Network needs in order to reason about, rather than merely classify over, the textual content.

The rest of this chapter describes the schema that implements that intuition, the leakage-prevention strategy that keeps the schema from accidentally leaking the outcome, and the text-embedding and scalar features attached to each node.

4.2 Graph Schema

The graph is constructed as a **full-star global graph**: every case is a local "star" of text and entity nodes anchored on a CASE node, and a selected set of entity types are *shared* across cases so that stars of different cases are stitched into one connected component.

4.2.1 Node Types

The reasoning-focused graph keeps **17 node types** in three categories:

- **The case node.** A single CASE node per case, whose attributes store the outcome label, bucket, filing and decision dates (used for temporal gating), and bookkeeping metadata.
- **Text nodes.** Six types – PREAMBLE, FACTS, ARGUMENTS, PETITIONER_ARGUMENTS, RESPONDENT_ARGUMENTS, and OTHER_LAWYER_ARGUMENTS – each carrying a `bge-m3` text embedding of the cleaned, leakage-filtered span they represent (Section 4.4).
- **Entity nodes.** Ten types: PETITIONER, RESPONDENT, PETITIONER_LAWYER, DEFENCE_LAWYER, LAWYER, COURT, JUDGE, STATUTE, PROVISION, and PRECEDENT.

Four auxiliary NER types that OpenNyAI produces – ORG, GPE, DATE, and CASE_NUMBER – are explicitly *removed* from the reasoning-focused graph (`FORBIDDEN_NODE_TYPES` in the updated graph policy). They carry little predictive signal for the outcome and would otherwise inflate the node count and, worse, introduce weak, noisy edges that dilute message passing.

Their removal is one of the key differences between the reasoning-focused variant used in this thesis and the original baseline schema.

4.2.2 Cross-Case Sharing

Whether two cases share a node, or each hold a private copy of one, determines whether a GNN can use that node as a cross-case conduit. The policy (`reasoning_graph_policy.py`) is deliberate:

- **Shared across cases**: STATUTE, PROVISION, and PRECEDENT. These are the *law itself* – citing IPC 420 in one case and IPC 420 in another is meant to be the same object, and sharing the node is what lets the GNN propagate information along the citation.
- **Local to each case**: everything else, including COURT, JUDGE, and all LAWYER variants. These are *identity proxies*: if a particular judge or a particular bench has a historical win/loss bias, sharing the node would let the GNN recover that bias as a free shortcut to the outcome – exactly the kind of spurious signal the experimental design in Chapter 6 is built to prevent. Keeping them local makes the model reason about the law, not about the docket.

4.2.3 Edge Types

The schema defines a concrete set of typed edges. The core structural edges are:

- `case--has_preamble/has_facts/has_arguments--*text*`: attach each text node to its case.
- `case--has_petitioner_arguments/has_respondent_arguments/has_other_lawyer_arguments--*text*`: attach party-specific argument blocks.
- `case--has_petitioner/has_respondent--*party*`, `case--heard_in--court`, `case--decided_by_bench--judge`, `case--has_petitioner_lawyer/has_defence_lawyer/has_lawyer--*counsel*`.

- `arguments--cites_statute--statute`, `arguments--cites_provision--provision`, `arguments--cites_precedent--precedent`: the citation backbone, attached to the ARGUMENTS node rather than to the case so that the citation sits in its legal-discourse context.

- `provision--belongs_to_statute--statute`: stitches the provision hierarchy into the graph so that a citation to Section 482 is two hops from IPC, not disconnected from it.

- `petitioner_lawyer--citation--petitioner_arguments`, `defence_lawyer--citation--respondent_arguments`, `lawyer--citation--other_lawyer_arguments`: connect each counsel to the argument block they are responsible for.

- `petitioner--is_party_in_arguments--petitioner_arguments`, `respondent--is_party_in_arguments--respondent_arguments`: tie parties to their own argument blocks.

All edges are added with their reverse counterparts (`include_reverse_edges`), and the final graph is passed through `torch_geometric.to_undirected` so that HGT's typed message passing can flow in both directions.

Shortcut edges that are intentionally removed. An earlier version of the schema added edges that let entities bypass the arguments node – e.g. `statute-used_in_arguments-arguments`, `judge-presided_arguments-arguments`, `petitioner/respondent-is_party_in_arguments-arguments` (to the *shared*, un-partitioned ARGUMENTS node). The updated policy *forbids* these edges (`FORBIDDEN_EDGE_TYPES`). Keeping them would have created short paths from identity proxies to the case representation that contribute little reasoning value but a lot of spurious correlation; the thesis uses the reasoning-focused schema for exactly this reason.

4.2.4 Attributes

Every CASE node carries a binary outcome label (`+1` win, `-1` loss) plus its bucket membership and the dates needed for temporal gating. Every text

node carries the raw text (for downstream inspection only; the model never sees it directly) and the pre-computed embedding. Every entity node carries its canonical name and the canonicalisation metadata needed to reproduce the merge. Edges are attributed with their relation type, which is what the HGT layers use to index their per-relation parameters (Chapter 6).

4.2.5 Visualizing the Graph Structure

To provide a concrete visual intuition of the graph schema described above, we present diagrams of a single case graph and a multi-case subgraph.

Figure 4.1 illustrates the local "star-graph" structure surrounding a single CASE node. The node serves as an anchor connected to its text components (e.g., Preamble, Facts, Arguments) and its identity entities (e.g., Court, Judge, Lawyers). From the Arguments node, further citation edges branch out to Statutes and Precedents.

Importantly, these outward citation edges maintain a strict chronological hierarchy: a newer case will only form a citation edge to a Precedent if precedent_year $<$ case_year, and to a Statute if statute_year $\leq$ case_year. This temporally-directed structure explicitly ensures that older cases cannot structurally link forward in time to newer artifacts.

Figure 4.2 demonstrates the cross-case sharing mechanism. To prevent spurious correlation (docket bias leakage), nodes like Courts and Judges are kept private to each case. In contrast, legal entities such as Statutes, Provisions, and Precedents are shared across cases, forming the cross-case conduits that allow the GNN to perform structural and legal reasoning without directly observing the outcome proxy.

4.3 Leakage Prevention Strategy

Legal judgment prediction has a notorious failure mode: models that appear to predict the outcome are actually *reading* it. The construction pipeline defends against this at three distinct points.

1. Field-level removal. The cleaning stage drops a fixed set of `FORBIDDEN_TOP_LEVEL_FIELDS` from the case JSON before any graph is built: `case_outcome_label`, `case_outcome_score`, `llm_case_outcome`, `decision_text`, `rpc_texts`, `short_explanation`, and

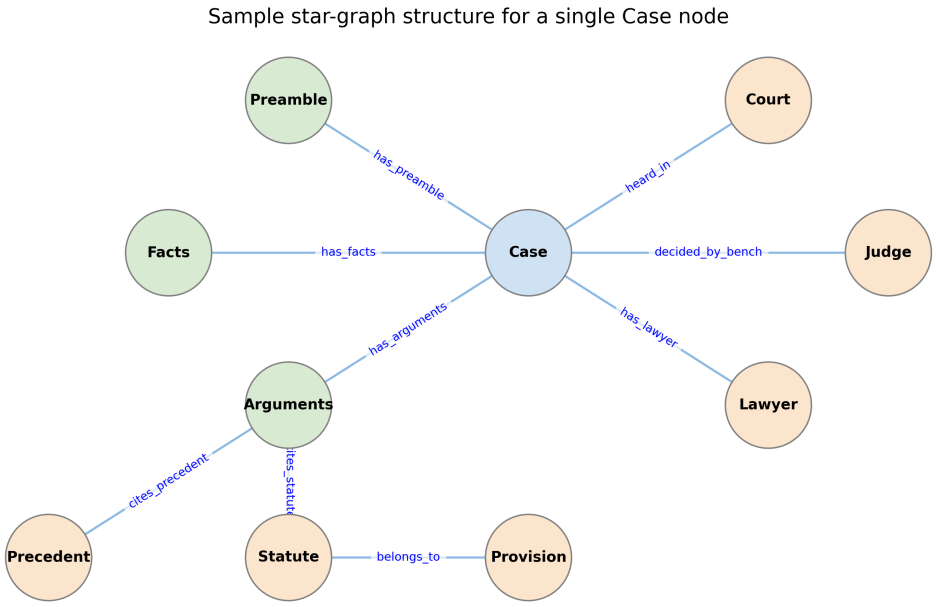


Figure 4.1: Sample star-graph structure for a single Case node. Various text blocks feature as text nodes, while details like Court and Judge form local entity nodes. The Arguments node branches out to cite shared legal concepts like Statutes and Precedents.

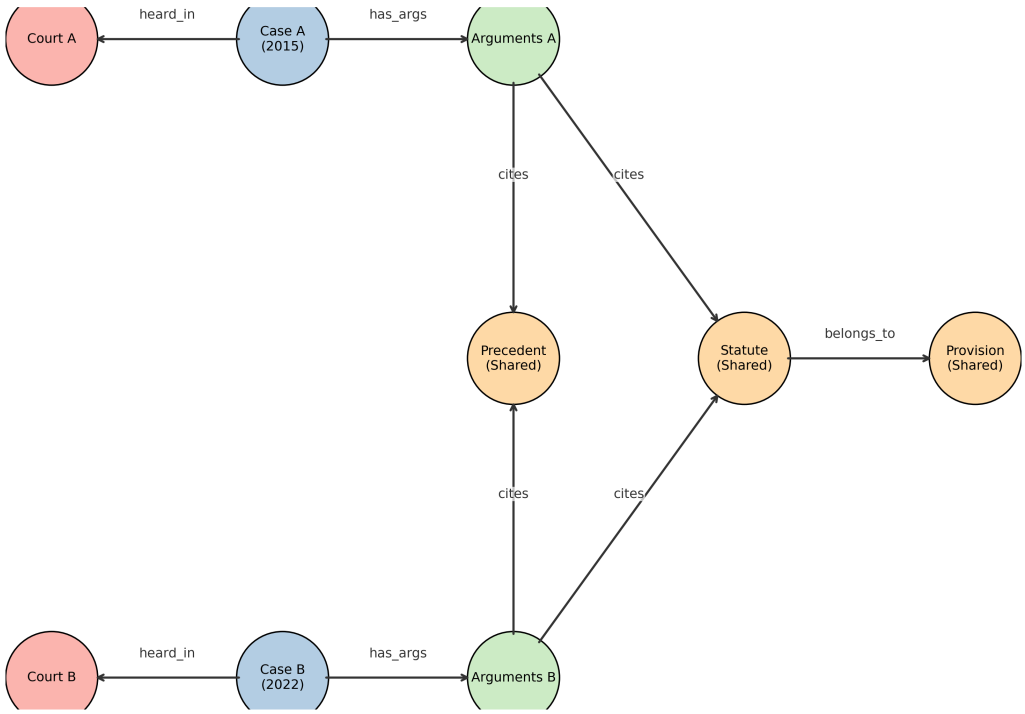


Figure 4.2: Cross-connection of cases via shared nodes. Each case maintains its own local representations for identity entities to prevent shortcut leakage. Shared nodes like Statutes and Precedents act as legally-sound cross-case conduits for message passing.

`raw_model_response`. It also drops any `raw_result.summary.decision` block produced by the OpenNyAI summariser.

2. Rhetorical-role filtering. The `FORBIDDEN_ANNOTATION_LABELS` set ({RPC, RLC}) plus a label-string blacklist (`decision`, `disposition`, `operative`, `order`, `rpc`) causes any annotation in those classes to be removed from the summary before the text nodes are built. Because RPC is where the court actually pronounces the result, this is the single most important filter: without it, the ARGUMENTS text node would contain the verdict verbatim.

3. Phrase-level masking. A list of `DEFAULT_LEAKAGE_PHRASES` – *"petition stands disposed of"*, *"appeal is allowed"*, *"set aside"*, *"quashed"*, and so on – is compiled into case-insensitive, whitespace-tolerant regex patterns and applied as a sub-sentence scrubber across every retained text field. Every match is logged in a per-case `leakage_audit`, and the phrase itself is replaced with a `[LEAKAGE_MASK]` token. This catches residual decision-bearing fragments that the RR filter missed because they were embedded inside a FACTS or ARGUMENTS sentence rather than tagged as RPC.

4. Temporal gating on shared entities. Sharing a PRECEDENT or STATUTE node across cases would leak future information if a precedent decided in 2022 were attached to a case from 2015. The graph builder therefore enforces a temporal rule inside `case_star_builder`: a precedent citation edge is added only when precedent_year $<$ case_year, and a statute citation edge only when statute_year $\leq$ case_year. This keeps the cross-case conduit legally honest.

5. Audit trail. Every case carries a `leakage_audit` structure recording the fields dropped, the annotations removed, the decision text removed, and every phrase-level mask. The audit is kept with the graph cache so that any future claim of leakage can be *disproved*, not merely argued against.

The combined effect of these five gates is that the text a GNN layer eventually sees through its text-node features is a pre-decision representation of the dispute – who the parties are, what happened, what each side argued, and what law they cited – with the decision itself systematically withheld.

4.4 Text Embedding and Feature Extraction

4.4.1 The `bge-m3` Text Encoder

All text nodes are featurised with `BAAI/bge-m3`, a multilingual sentence-transformer with a 1024-dimensional output, run through the `sentence_transformers` library. The choice of `bge-m3` is driven by two concrete requirements: (i) Indian legal text mixes English with Hindi and regional-language transliterations, Latin-script legal abbreviations, and citation fragments, and `bge-m3`'s multilingual training data handles this noticeably better than English-only encoders; and (ii) the model is strong in retrieval settings, which aligns with the graph's use of embeddings as comparison features across cases.

Configuration. The encoder is run with `batch_size=16`, `max_length=512`, on GPU, and with multi-process mode enabled. When multiple CUDA devices are visible the encoder spins up one worker per device, distributes the text list, and merges the results, which is what makes encoding millions of argument and facts spans tractable in wall-clock terms.

Hierarchical chunking for long spans. Legal judgment *Facts* and *Arguments* sections routinely exceed the model's 512-token window. For texts longer than the configured character limit, the encoder splits them into overlapping, sentence-boundary-aligned chunks, encodes each chunk independently, and mean-pools the resulting vectors. This preserves a consistent 1024-dimensional representation per text node regardless of input length, while avoiding the information loss of a hard truncation.

Caching. Embeddings are cached per namespace with a deterministic hash of the input ids, texts, and encoder configuration. Re-running the graph build with an unchanged encoder loads cached vectors in seconds; changing any encoder-sensitive field (`backend`, `model_name`, `max_length`, `hierarchical_chunk_chars`) invalidates the cache. A `hashing` fallback encoder is available for local debugging when GPU resources are unavailable.

4.4.2 Node Feature Assembly

For each node type the final per-node feature vector is the concatenation of:

- the 1024-dim `bge-m3` embedding of the node's canonical text (for text nodes, that is the full span; for entity nodes, that is the canonical name);
- scalar node-level features: local frequency in the case, section-of-first-mention indicators (`seen_in_preamble`, `seen_in_arguments`), type one-hot, and for shared entity types the log global frequency across the corpus;
- for the Case node, bucket one-hot and the outcome label is attached only as the supervision target, never as a feature.

Because different node types carry different feature dimensionalities, the HGT model used in Chapter 6 begins with a per-node-type linear projection (`input_projections`) that maps all features into a common hidden dimension before the first heterogeneous message-passing layer. This is the mechanism that lets the graph remain genuinely heterogeneous at the feature level while still admitting a uniform message-passing computation.

Chapter 5

Graph Exploration and Structural Insights

5.1 Why Explore the Graph Before Prediction?

It is tempting to treat the graph built in Chapter 4 as a black box that is fed into a GNN and evaluated by a single F1 number. That view is wrong for two reasons.

First, *the graph is the dataset.* In a flat-text pipeline the dataset is a set of labelled documents; here the dataset is a typed, relational object with cross-domain structure, and any claim a later chapter makes about the model's behaviour presupposes a concrete understanding of that structure. Whether the model's accuracy is driven by genuine cross-case reasoning or by a few dominant hub statutes, whether the five legal domains are actually five or mostly one, whether the per-case subgraph has the connectivity a message-passing layer needs – none of these questions can be answered by looking at prediction results alone.

Second, *leakage-safety claims are structural claims.* The leakage prevention strategy in Section 4.3 is only as convincing as the structural evidence that the remaining graph is still dense enough to support learning. If removing RPC annotations, decision fields, and identity-proxy shortcuts left behind a near-disconnected graph, the downstream accuracy would be either implausibly high (still leaking) or implausibly low (now uninformative). The exploration stage is where we verify that neither is the case.

This chapter therefore treats the graph as an object of study in its own

right. Section 5.2 summarises the overall structure; Section 5.3 breaks it down per legal domain; Section 5.4 distils the conclusions that feed forward into Chapter 6.

5.2 Graph Statistics and Structural Analysis

5.2.1 Overall Graph Statistics

The final reasoning-focused graph, assembled across all five legal domain buckets, contains **880,176 nodes** and **1,977,295 edges**. The average node degree is **4.49**; the maximum degree is **29,602**, attained by the IPC statute node.

At the per-bucket level (before cross-bucket sharing is expanded into the unified view the GNN trains on), the five buckets have broadly comparable case counts but markedly different internal densities:

Bucket	Cases (files)	Nodes	Edges
Family & Matrimonial	15,307	97,150	3,660,068
Financial Fraud	14,616	113,240	7,869,952
Land & Property	14,052	126,962	9,210,612
Motor Accidents	15,237	125,234	7,165,461
Sexual Offences	14,597	106,662	7,123,225
Cross-bucket unified	**73,809**	**485,540**	**27,243,141**

Table 5.1: Per-bucket and cross-bucket graph statistics. The cross-bucket graph is denser than the sum of the per-bucket graphs because shared STATUTE, PROVISION, and PRECEDENT nodes acquire edges from every bucket that cites them.

Family-matrimonial is the sparsest bucket by edges-per-case (≈ 239), and land-property is the densest (≈ 655); the difference is driven mostly by the number of distinct provisions and precedents cited per case in each domain.

5.2.2 Node Type Distribution

The node-type breakdown of the unified graph is strongly skewed towards **parties and precedents**:

Node type	Count	Share
Case	72,735	8.3%
Precedent	255,148	29.0%
Respondent	192,147	21.8%
Petitioner	171,084	19.4%
Lawyer	73,645	8.4%
Provision	43,504	4.9%
Court	38,799	4.4%
Statute	21,754	2.5%
Judge	11,360	1.3%

Table 5.2: Distribution of the 880,176 nodes by type in the unified reasoning-focused graph.

Understanding which of these node types are *shared* across cases versus *local* to a single case is central to understanding how information can propagate through the graph. Shared nodes are the mechanism by which a GNN can transfer knowledge from one case to another; local nodes contribute features only to the case they belong to. Figure 5.1 visualises this split per bucket.

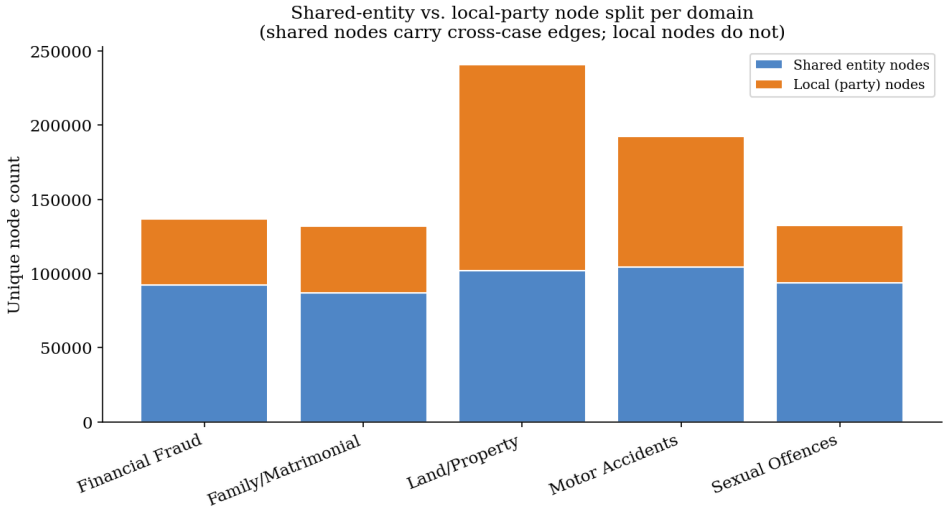


Figure 5.1: Shared versus local node counts per legal domain bucket. Party nodes (PETITIONER, RESPONDENT, most LAWYER nodes) are almost entirely local — a party in one case rarely recurs in another. Statutes, provisions, courts, and frequently-cited precedents form the shared layer that is the graph's cross-case connective tissue.

The figure confirms that the split is sharp and legally interpretable. Party nodes are almost entirely local: a petitioner or respondent in one matter rarely appears in another unrelated case. Statutes, provisions, courts, and frequently-cited precedents, by contrast, form a small but densely shared layer. This is the layer through which all cross-case message passing must flow. The implication is that removing party nodes from the shared layer—as the leakage-safe design does for judges and specific lawyers—does not destroy cross-case connectivity; the statute and precedent layer remains fully intact and is the genuine information carrier. The statute side of the graph is small in node count but, as the next subsection shows, carries disproportionate structural weight.

5.2.3 Degree Distribution and Hub Entities

The graph is strongly hub-dominated. The top-12 nodes by degree are all statutes, provisions, or courts – exactly the node types the schema shares across cases:

Node	Type	Degree	Buckets bridged
IPC	Statute	29,602	5
CR.P.C.	Statute	18,704	5
Supreme Court	Court	17,294	5
Apex Court	Court	14,242	5
Indian Penal Code	Statute	12,009	5
High Court of Judicature at Allahabad	Court	8,833	5
Section 482	Provision	8,732	5
Section 506	Provision	8,258	5
I.P.C.	Statute	7,051	5
Section 420	Provision	6,285	5
Section 323	Provision	6,002	5
Constitution of India	Statute	5,950	5

Table 5.3: Top hub entities by degree, with the number of buckets they bridge. Every one of the top 12 hubs appears in all five legal domains.

The PageRank ranking tells the same story from a different angle: IPC (PR = 0.0114), Supreme Court (0.0082), CR.P.C. (0.0069), and Apex Court (0.0067) dominate the top of the distribution. That every one of the top hubs bridges all five buckets is not an accident of the numbers – these are the statutes and courts every Indian criminal and civil proceeding touches – but it has a concrete modelling consequence: these hubs are where cross-case message passing actually happens.

To understand whether this hub dominance is a localised anomaly or a systemic structural feature, Figure 5.2 plots the complementary cumulative degree distribution (CCDF) on a log-log scale.

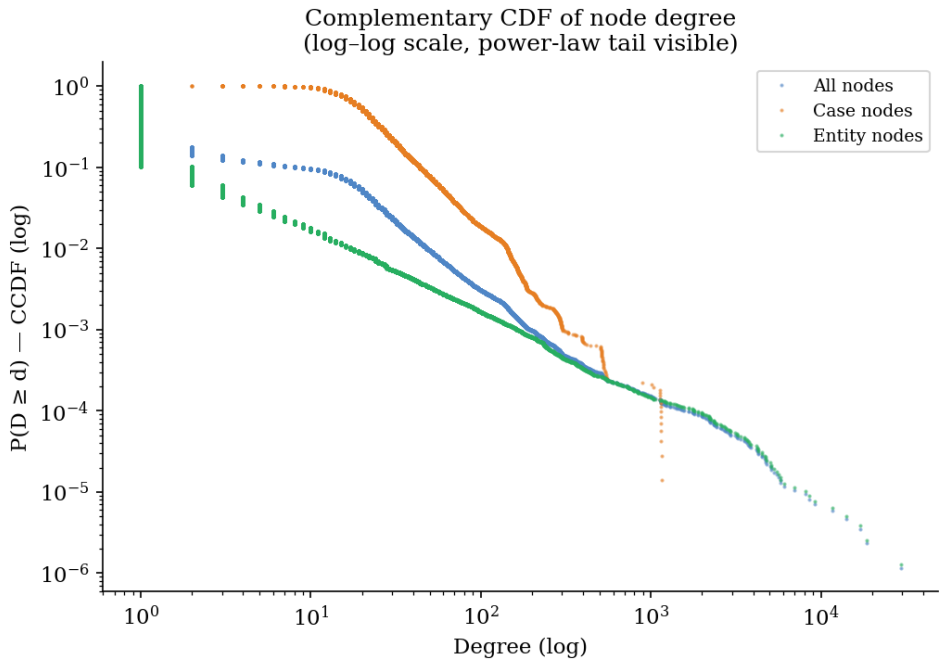


Figure 5.2: Complementary cumulative degree distribution (CCDF) of the unified graph on a log-log scale. The approximately linear decay is the hallmark of a heavy-tailed degree distribution: most nodes have very low degree, while a small number of hub nodes accumulate orders of magnitude more edges than the average, confirming that hub dominance is a systemic structural feature rather than an artefact of a few outliers.

The approximately linear decay in this plot is the signature of a heavy-tailed, scale-free-like degree distribution. The vast majority of nodes sit at degree one or two, while a thin upper tail extends to degrees of tens of thousands. This two-regime structure – a small set of super-hubs plus a long tail of single-case leaves – has a direct modelling consequence: message passing aggregates over all neighbours, so hub nodes receive and broadcast information from a disproportionately large fraction of the graph. The cross-case reasoning capacity of the GNN is therefore structurally concentrated in a handful of statute and court nodes. The "connectivity score" in Section 5.2.6 is designed to quantify how well each individual case is wired into this hub layer.

5.2.4 Cross-Domain Bridge Analysis

The buckets are not isolated. Table 5.3 already shows that the top hubs are bridges by construction; the full bridge analysis confirms that the connective tissue between domains is dominated by the general criminal-procedure and penal-code statutes, with domain-specific bridges layered on top:

- IPC, CR.P.C., Indian Penal Code, Constitution of India, and Supreme Court / Apex Court appear in all five buckets and together account for the majority of cross-bucket edges.
- Domain-specific statutes bridge fewer buckets but at higher within-bucket intensity: POCSO Act appears in four buckets but is heavily concentrated in sexual-offences; Motor Vehicles Act, 1988 appears in all five but is concentrated in motor-accidents; Land Acquisition Act, 1894 is concentrated in land-property.
- Provisions like Section 482, 506, 420, and 323 are multi-bucket because they sit in the CrPC and IPC, which are themselves multi-domain.

To assess whether high degree and broad bridging are correlated or distinct properties, Figure 5.3 plots the top hub entities by degree alongside the number of buckets they appear in, combining both dimensions into a single view. The figure reveals that the two dimensions are correlated but not identical: some high-degree nodes are concentrated within a single domain (heavily-cited domain-specific provisions), while the universal bridges — IPC, CR.P.C., Supreme Court — sit at the top of both rankings simultaneously. This distinction matters directly for cross-domain transfer: only the universally shared hubs actually mediate information flow between all five buckets during message passing. A high-degree domain-specific hub, by contrast, influences prediction accuracy within its own bucket but contributes nothing to cross-bucket generalisation.

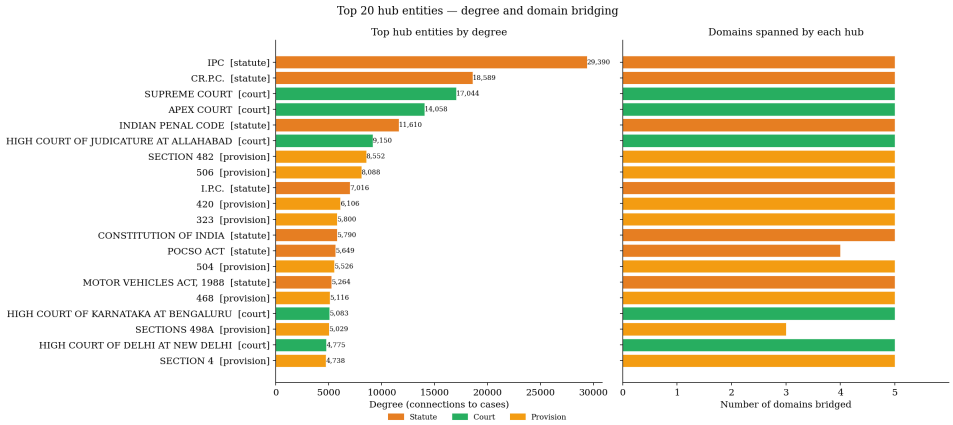


Figure 5.3: Top hub entities ranked by degree, with the number of legal domain buckets they bridge encoded by colour or marker size. High degree and broad bridging are correlated but not equivalent: the universal bridges (IPC, CR.P.C., Supreme Court) top both rankings, while domain-specific high-degree hubs appear in fewer buckets.

5.2.5 Cross-Bucket Entity Sharing Matrix

Knowing which individual nodes are hubs is useful; knowing how many entities are shared *between each pair of domains* is a different and complementary question. If two domains share very few entities, cross-domain transfer between them is structurally impossible — there are no shared nodes for message passing to flow through. If two domains share nearly all entities, they are functionally the same graph and a separate bucket structure adds no value. The cross-bucket sharing matrix in Figure 5.4 quantifies this pairwise sharing at the canonicalised entity level across the full graph.

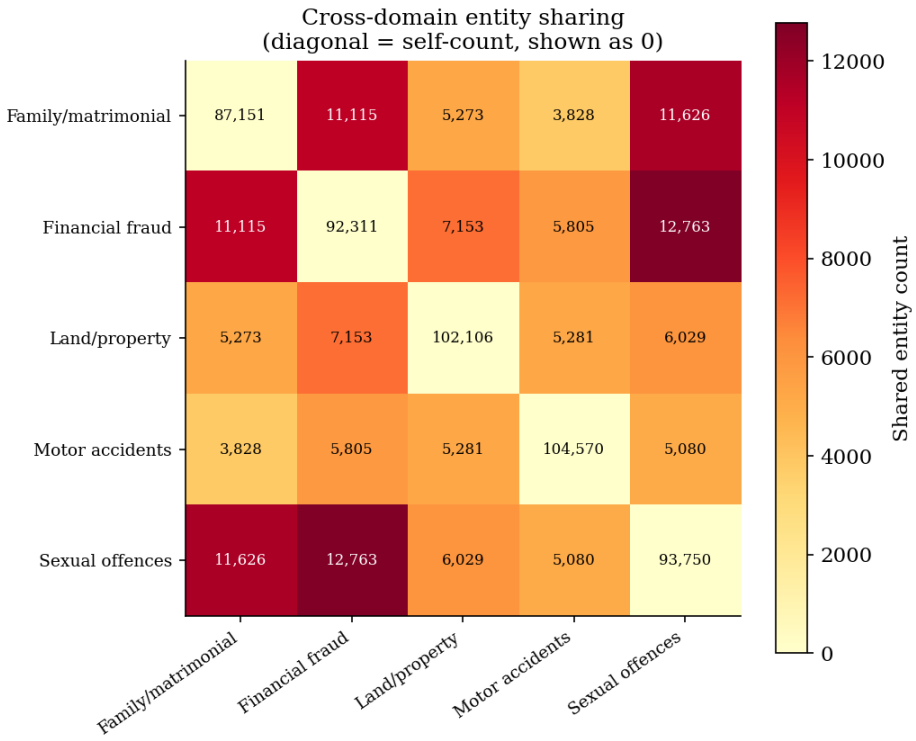


Figure 5.4: Cross-bucket entity sharing matrix for the full graph. Each cell shows the number of canonicalised entities shared between two legal domain buckets. Darker values indicate more sharing. The three IPC/CrPC-driven buckets — family-matrimonial, financial-fraud, and sexual-offences — share heavily with each other; land-property is more weakly connected to the rest via its distinct statutory backbone.

Three patterns are visible in the matrix:

1. Strong pairwise sharing among *Family & Matrimonial*, *Financial Fraud*, and *Sexual Offences* — all three are IPC/CrPC-driven and share the same high-degree statute and provision nodes.

2. Moderate sharing between those three and *Motor Accidents*, mediated by the general procedural statutes and by the Supreme Court and High Court nodes that every appellate domain shares.

3. Comparatively weaker sharing with *Land & Property*, which is anchored by its own statutory backbone (Land Acquisition Act 1894, Constitution

of India) and consequently has fewer canonical entities in common with the criminal-procedure-dominated domains.

This pattern is exactly what a cross-domain transfer claim needs to be credible. The domains are related enough that a shared graph representation should carry useful signal across all five buckets; they are different enough that each bucket retains its own statutory fingerprint and that a single-domain classifier should not trivially dominate a cross-domain one. The moderate but not extreme sharing level motivates the design choice of a unified cross-bucket graph rather than five separate per-domain graphs: there is sufficient shared structure to benefit from joint training, but sufficient domain separation to make that benefit non-trivial.

5.2.6 Connectivity Score Distribution

The hub analysis above characterises individual nodes; a complementary question is how well each *case* as a whole is wired into the shared-entity layer. A case that cites hub statutes and widely-cited precedents has many high-degree shared neighbours and can therefore receive rich aggregated information from a large neighbourhood during message passing. A case that sits primarily on rare or single-occurrence entities is structurally isolated and receives little cross-case signal. To make this distinction concrete, a **connectivity score** is assigned to each case as a bucket-normalised measure of how well it is connected to the rest of the graph through its shared-entity neighbours (statutes, provisions, precedents). This score is important both as a structural diagnostic and as a predictor of per-case modelling difficulty: a GNN layer's information gain on a low-connectivity case is, by construction, structurally bounded.

Figure 5.5 shows the per-bucket connectivity score distribution as a violin plot, which reveals not just the central tendency but the full distributional shape.

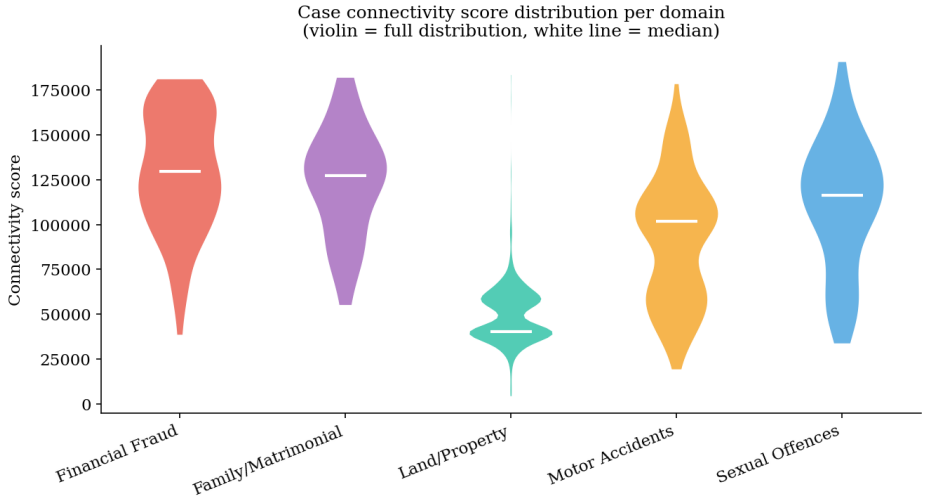


Figure 5.5: Per-bucket connectivity score distribution as violin plots. Land-property and financial-fraud show wider distributions with higher medians — their cases cite more diverse authorities and are better connected to the graph's shared layer. Motor-accidents is sharply concentrated at low connectivity values, reflecting repeated citation of the same narrow provision set. Within-bucket variance predicts case-level prediction difficulty.

Land-property and financial-fraud show wider, higher-median distributions: their cases cite diverse authorities and are therefore well-connected to the hub layer. Motor-accidents shows a narrow distribution concentrated at low values, reflecting the domain's repeated citation of the same small provision set (Motor Vehicles Act, Section 166). The within-bucket variance is itself informative: even within a single domain, cases span a substantial connectivity range, meaning that prediction difficulty is heterogeneous at the case level and cannot be attributed to domain alone. The ablation pattern in Chapter 6 is consistent with this expectation: cases in the low-connectivity tail of any bucket show systematically higher error rates. The connectivity score is also used by the Graph Visualiser (`GRAPH_VISUALISER/`) to rank cases when sampling Hub-Network and Case-Connection views.

5.3 Per-Domain Corpus Analysis

The structural view above aggregates across buckets; the per-domain view is where the legal character of each bucket shows up.

5.3.1 Case Length and Sentence Distributions

Case length varies substantially across buckets, and that variance is not merely a descriptive curiosity — it directly determines how text features are constructed for the GNN. Long documents require chunked embedding strategies; short documents can be embedded whole. Understanding the distribution shapes the chunking configuration and lets us flag which domains will produce the most embedding approximation.

Figure 5.6 shows the sentence-count distribution per bucket across the full corpus.

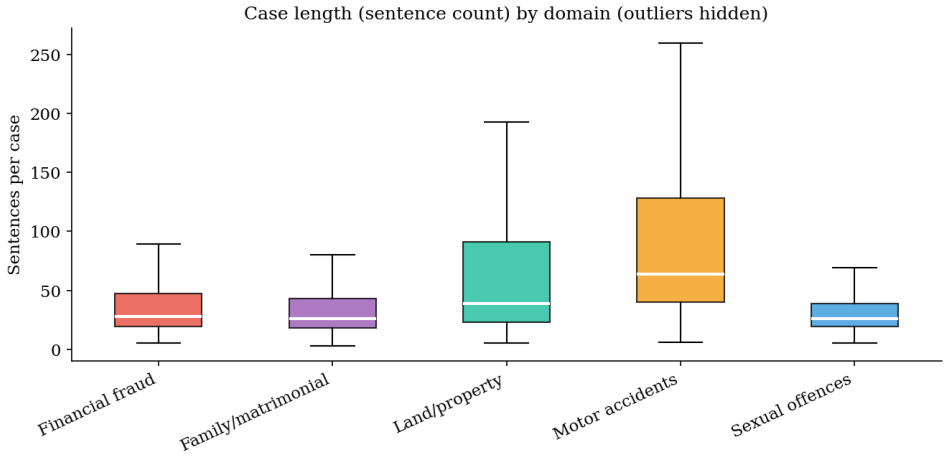


Figure 5.6: Sentence-count distribution per legal domain bucket across the full corpus. Land-property and financial-fraud have the heaviest right tails (long, document-heavy judgments); motor-accidents is concentrated at low sentence counts (short tribunal orders). These distributions directly drove the per-bucket chunking strategy for `bge-m3` embedding.

The distributions confirm the ordering expected from domain knowledge. Motor-accidents judgments are typically short and tribunal-style, concentrated

well below 100 sentences. Land-property matters are long and document-heavy (deeds, surveys, possession history), with a pronounced right tail extending into several hundred sentences per case. Financial-fraud and family-matrimonial sit between these extremes. Sentence-count distributions follow the same ordering as overall document length and are the primary driver of the hierarchical-chunking configuration in Section 4.4: long-tail FACTS and ARGUMENTS sections in land-property and financial-fraud buckets exceed the `bge-m3` context window and are mean-pooled across overlapping chunks, with the chunk boundary set per-bucket based on the 90th-percentile sentence count.

5.3.2 Entity Richness Per Domain

Entity richness — the number of unique canonicalised entities per case — is a per-domain structural property that predicts both graph density and expected modelling difficulty. A domain where every case cites a wide variety of distinct statutes, provisions, and precedents will produce a denser, more richly interconnected subgraph; a domain where cases repeatedly cite the same small set of authorities will produce a sparser subgraph with lower information diversity per case. Figure 5.7 plots entity richness alongside mean case degree for each bucket, providing a direct comparison of both dimensions.

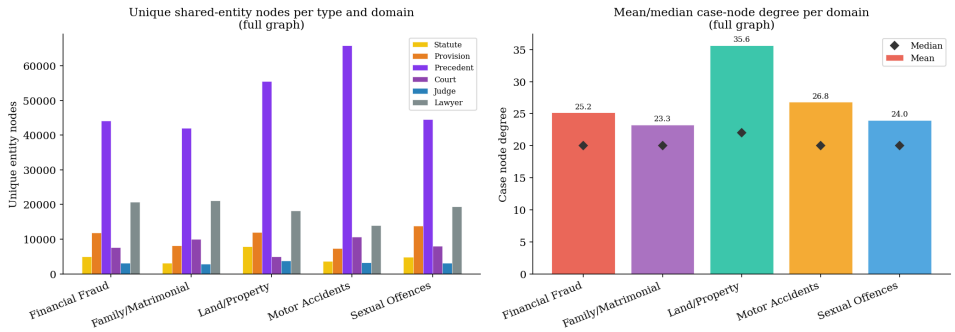


Figure 5.7: Entity richness (unique canonicalised entities per case) and mean case degree per legal domain bucket. Land-property and financial-fraud are both entity-rich and highly connected; motor-accidents is entity-poor and low-degree. The correlation between richness and degree confirms that these are domain-level phenomena rather than noise effects.

The figure confirms the expected ordering. Land-property and financial-fraud score highest on both axes: their cases cite many distinct statutes, provisions, and precedents, and are therefore well-connected to the graph's shared-entity layer. Motor-accidents scores lowest on both: it repeatedly cites the same narrow provision set (Motor Vehicles Act 1988, Section 166) and produces cases with comparatively few graph edges. The correlation between richness and degree is expected — a case that cites many distinct authorities will naturally have more edges — but the per-domain consistency confirms that this is a domain-level structural phenomenon rather than variance driven by outlier cases. Domains with high richness and degree will benefit more from multi-hop message passing than sparse domains; the ablation in Chapter 6 tests this prediction directly.

The PageRank ranking within each bucket makes the domain fingerprint concrete:

- **Family & Matrimonial**: IPC, CrPC, Sections 498A, 482.
- **Financial Fraud**: IPC, CrPC, Sections 420, 468, 471.
- **Land & Property**: Supreme Court, Land Acquisition Act 1894, Constitution of India.
- **Motor Accidents**: Motor Vehicles Act 1988, Supreme Court, Apex Court, Section 166.
- **Sexual Offences**: IPC, CrPC, POCSO Act, Section 506.

The top-5 per bucket is a near-perfect fingerprint of the underlying legal domain, which is a reassuring sanity check: the canonicalisation and the NER are recovering legally meaningful primary citations, not surface noise.

The table above tells us *which* entities are important; it does not tell us *how* they are organised relative to each other within the bucket. A domain where the top-ranked entities form a tight, star-shaped cluster around a single authority behaves very differently from one where the top entities form a distributed, multi-centred network — even if the ranked lists look superficially similar. Figure 5.8 makes this geometry visible by showing the entity co-occurrence network within each bucket restricted to its top-25 PageRank entities. This is the strongest diagnostic available for whether the domain fingerprints recovered by canonicalisation are structurally coherent, not just statistically high-ranking.

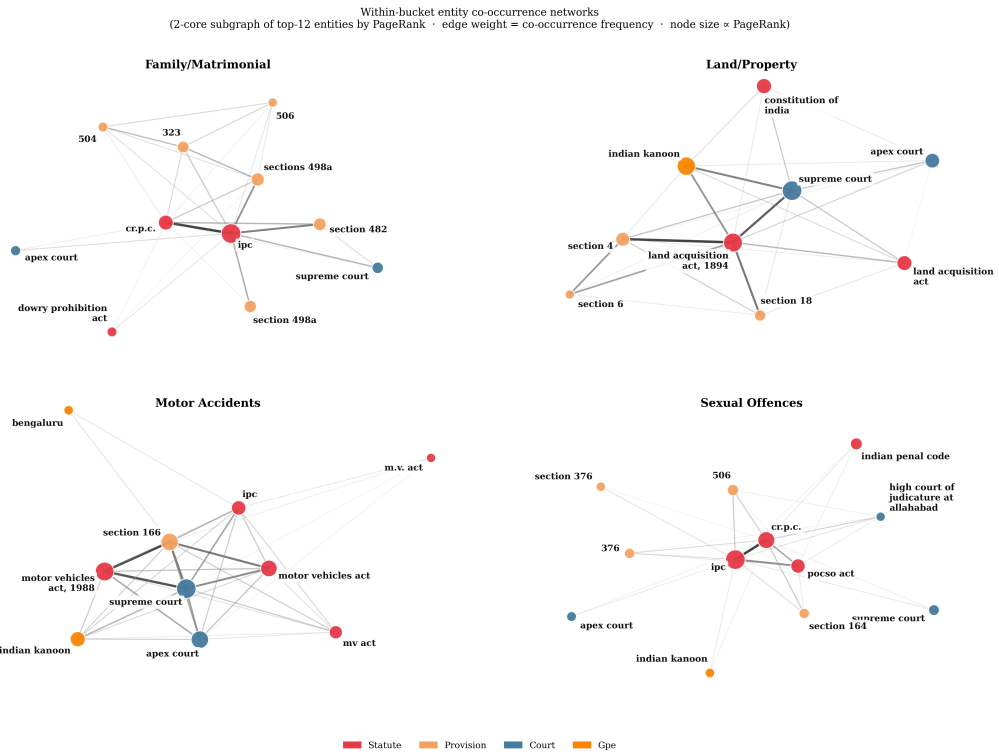


Figure 5.8: Within-bucket entity co-occurrence networks for four legal domains, each restricted to the top-25 PageRank entities in that bucket. Node size is proportional to PageRank score. The figure reveals not just which entities rank highest but how they are geometrically organised: tight criminal-procedure stars in family-matrimonial and sexual-offences, a distributed multi-centred structure in land-property, and an extremely compact low-diversity backbone in motor-accidents.

The networks reveal that the domains differ not only in their top entities but in their *network geometry*, and that the differences are legally interpretable. Family & matrimonial and sexual offences both remain visibly IPC/CrPC-centred, yet they bifurcate into distinct provision clusters almost immediately below the top hub: Section 498A, 482, and dowry-related citations form the inner ring in family-matrimonial, while POCSO, Section 376, and Section 363 dominate in sexual offences. The two domains share the same criminal-procedure spine but are routed through different specialised provisions —

exactly the relationship predicted by the entity-sharing matrix (Figure 5.4).

Land-property is geometrically distinct from both: its network is spatially spread with several semi-central statutory and constitutional nodes rather than one overwhelmingly dominant criminal-procedure core. The LAND ACQUISITION ACT 1894 and the CONSTITUTION OF INDIA sit as co-equal secondary hubs alongside SUPREME COURT, forming a multi-centred structure that reflects the domain's diverse statutory and constitutional basis and is consistent with its higher entity richness shown in Figure 5.7.

Motor-accidents, by contrast, exhibits the tightest and most repetitive local backbone of all four domains, with MOTOR VEHICLES ACT 1988 and Section 166 forming an extremely compact star. The near-absence of peripheral nodes confirms that most motor-accidents cases enter the graph through the same narrow set of recurring authorities, leaving little structural diversity for multi-hop message passing to exploit. This visual finding is the network-geometry counterpart of the low connectivity score and low entity richness established in the preceding subsections.

Together, these bucket-level network snapshots make the domain-fingerprint claim visual rather than purely tabular, and provide structural grounding for the prediction chapter: domains with richer, more spread network geometry (land-property, financial-fraud) can be expected to benefit more from multi-hop message passing than domains with tight, low-diversity backbones (motor-accidents).

The within-bucket view, however, treats each domain in isolation. To understand how entities are *shared* or *segregated* across all five domains simultaneously, Figure 5.9 collapses the five independent networks into a single cross-domain graph. Each domain is represented as a large hub node placed at the vertex of a pentagon; the top-10 PageRank entities from each domain appear as satellite nodes, connected to whichever hubs they belong to. Entities that appear in the top-10 of two or more domains are drawn with a bold outline and linked by crimson cross-domain edges, making the inter-domain bridges immediately legible.

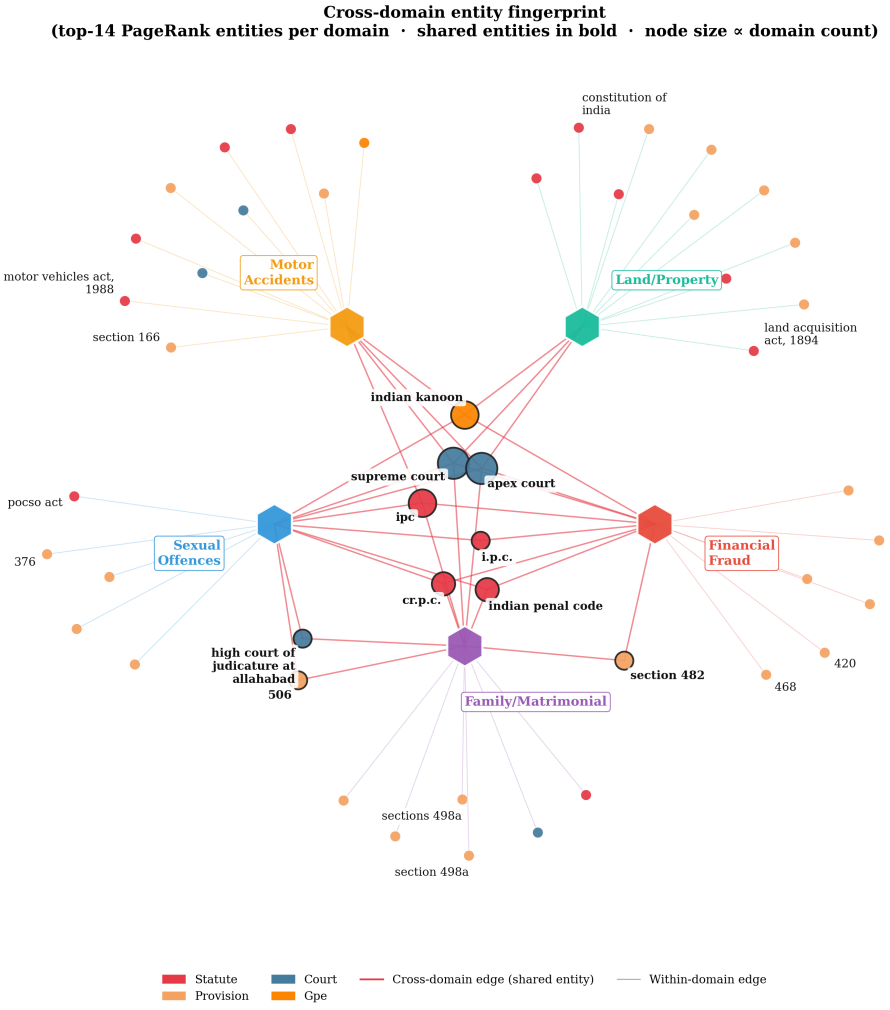


Figure 5.9: Cross-domain entity fingerprint: the top-10 PageRank entities from each of the five legal domains are placed around five domain-hub nodes arranged in a pentagon. Node colour encodes entity type; node size encodes the number of domains the entity appears in. Crimson edges connect entities that bridge two or more domains; domain-coloured edges connect exclusive entities to their single hub.

The cross-domain view reveals a clear three-tier bridge structure. At the widest tier, Supreme Court and Apex Court appear in the top-10

of all five domains — universal procedural anchors that carry interpretive weight regardless of substantive legal area. At the intermediate tier, IPC, CrPC, and Section 482 bridge the three criminal-procedure domains (Family/Matrimonial, Financial Fraud, and Sexual Offences) but are absent from the top tier of Land/Property and Motor Accidents — confirming that the criminal-procedure spine is real but domain-bounded. At the narrowest tier, domain-exclusive entities (Land Acquisition Act 1894 for land-property, Motor Vehicles Act 1988 and Section 166 for motor-accidents, POCSO for sexual offences) remain firmly segregated, appearing only in the satellite cluster of their single hub. This stratified bridge structure is precisely what a well-designed heterogeneous graph should exhibit: enough shared structure for cross-domain representation learning, but enough domain exclusivity for the GNN to recover meaningful per-domain fingerprints rather than collapsing all five into a single undifferentiated embedding.

5.3.3 Rhetorical Role Composition Per Domain

The rhetorical-role composition of the training corpus matters for two reasons. First, RPC and RLC annotations are the primary leakage vector and must be absent from the cleaned graph; verifying their absence domain by domain is a structural safety check, not just a logical one. Second, the relative sizes of the remaining blocks (Facts, Arguments, Preamble) determine the text-feature distribution the GNN trains on, and domain differences in those proportions explain part of the per-domain variation in embedding quality. Figure 5.10 shows the rhetorical role breakdown after leakage-safe filtering.

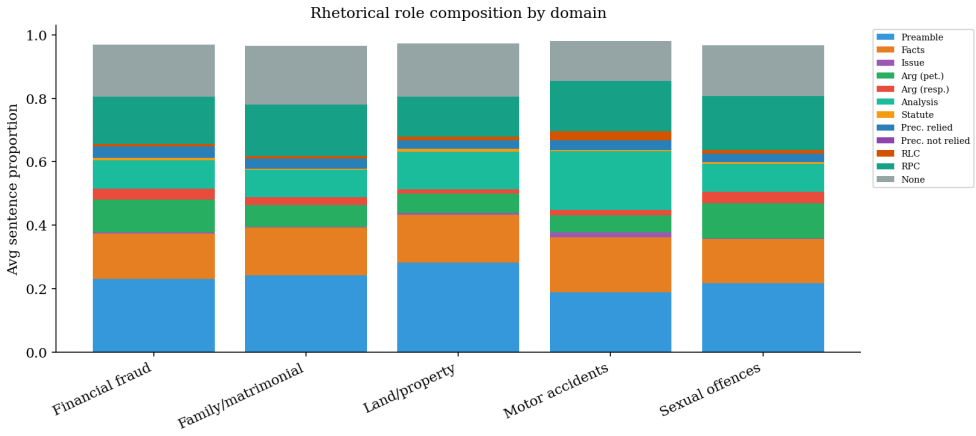


Figure 5.10: Rhetorical role breakdown per bucket after leakage-safe filtering. RPC and RLC annotations are intentionally absent, confirming the leakage gate is effective across all five domains. FACTS and ARGUMENTS dominate; the relative size of ARGUMENTS tracks case length, being largest in land-property and financial-fraud.

The composition after filtering is dominated by FACTS and ARGUMENTS, with a stable PREAMBLE share across domains. The relative size of the ARGUMENTS block tracks case length: it is largest in land-property and financial-fraud and smallest in motor-accidents, consistent with the sentence-count distributions in Section 5.3. The complete absence of RPC and RLC labels confirms that the leakage filter is operating correctly across all five buckets.

5.3.4 Outcome Distribution Per Domain

The outcome label distribution and its per-domain win/loss rates were analysed in detail as part of the pipeline audit in Chapter 3 (Figures 3.5 and 3.6). From the graph-exploration perspective, the relevant structural observation is that the class imbalance is *domain-dependent*: motor-accidents is the most win-skewed, financial-fraud and sexual-offences sit closest to balanced. This means the GNN's training signal is not uniformly imbalanced — different domains present different prior odds — which in turn means that per-domain calibration, rather than a single global threshold, is the appropriate evaluation design. The cross-bucket pooled win rate of roughly 60% to 40% is why

Chapter 6 reports macro-F1 alongside accuracy and ROC-AUC as co-primary metrics.

5.4 Key Structural Insights

Five insights carry forward into the prediction chapter:

1. **The graph is hub-dominated.** A small number of cross-bucket statutes and courts (IPC, CrPC, Supreme Court) are the spine of the graph; most other entities are long-tail leaves. Any claim the model makes about "cross-case reasoning" must be interpretable against this topology.

2. **Cross-domain sharing is real but uneven.** The four IPC/CrPC-driven buckets share heavily with each other; land-property partially detaches via its own statutory backbone. A cross-domain model should outperform a naive concatenation only if it can actually exploit the shared spine.

3. **Identity-proxy leakage is structural.** Judges, specific lawyers, and specific courts have bridge and betweenness scores high enough to make *sharing* them across cases unsafe; the decision to hold them local is vindicated by the bridge analysis.

4. **Connectivity score predicts modelling difficulty.** Low-connectivity cases have, by construction, fewer shared-entity neighbours to aggregate from; the per-domain accuracy pattern in Chapter 6 is consistent with connectivity being a first-order driver of case-level difficulty.

5. **The per-domain fingerprints are legally coherent.** Each bucket's top-PageRank entities are exactly what a lawyer would expect; this is the single strongest evidence that the pipeline's NER, canonicalisation, and rhetorical-role filtering together produce a legally faithful, not merely statistically clean, corpus.

Chapter 6

Prediction Model and Results

6.1 Why a Heterogeneous Graph Transformer?

The exploration in Chapter 5 made two demands of the model. First, the graph has *many node types* and each carries its own semantics – a STATUTE is not a CASE, and message-passing with a shared weight matrix would force the two to live in the same geometry. Second, the graph has *many edge types* and the structural role of an edge depends on its type: `cites_statute`, `heard_in`, and `belongs_to_statute` should not share aggregation weights.

The Heterogeneous Graph Transformer (HGT) satisfies both requirements. It maintains per-node-type projections, per-edge-type messages, and multi-head attention over the heterogeneous neighbourhood, which is what a typed message-passing layer needs to reason about the graph as the relational object it is rather than as a flat homogeneous approximation. Compared to the two natural alternatives – RGCN and HAN – HGT additionally learns the relative importance of different relations via attention rather than via a fixed per-relation weight, which turns out to matter on this graph because the predictive weight of (say) `arguments` $\rightarrow$ `cites_statute` $\rightarrow$ `statute` is not the same as `case` $\rightarrow$ `heard_in` $\rightarrow$ `court`.

6.2 Model Architecture

Input projection. Every node type is first mapped to a common hidden dimension by a per-type linear layer (`input_projections`), and a learnable type embedding (`type_embeddings`, initialised at $\mathcal{N}(0, 0.02^2)$) is added. This

preserves the heterogeneity of the input features (1024-dim `bge-m3` for text nodes, concatenated scalar features for entity nodes) while giving the message-passing layers a uniform geometry to work in.

Stacked HGT layers. Two `HGTConv` layers with four attention heads and hidden dimension 128 form the message-passing core. Each layer is wrapped in a residual connection with per-node-type `LayerNorm`, dropout ($p = 0.2$), and a ReLU non-linearity. Two layers is deliberate: the graph diameter between a CASE and a shared STATUTE is two hops (case $\rightarrow$ arguments $\rightarrow$ statute), so two layers is sufficient to move information between the two kinds of signal the model needs – the case's own textual evidence and the cross-case statutory neighbourhood.

Classifier head. The CASE node's final hidden representation is fed through a two-layer `MLPHead` producing the binary outcome logits.

Reference implementation. The architecture is parametric in `hidden_dim`, `num_layers`, `num_heads`, and `dropout`, and also supports a `HeteroConv`/SAGE fallback for ablation purposes; all experiments in this chapter use the HGT configuration above unless stated otherwise.

Training configuration. AdamW with learning rate 1×10^{-3} and weight decay 1×10^{-5}, cross-entropy loss with balanced class weights, 60 epochs with best-validation-F1 early stopping (patience 10). The loss and optimiser run on the full transductive graph; only the case-node indices are masked into train/val/test partitions. The `party_args_lr_decay` variant additionally applies a layer-wise learning-rate decay to the party-and-argument sub-module, which, as Section 6.6 shows, is the best-performing configuration.

6.3 Experimental Setup

Transductive protocol. Training is *transductive*: the full graph – including the test cases' text nodes and entity neighbourhoods – is visible to the model, but the test cases' *labels* are withheld through a case-node mask. This matches the real-world use case (a new judgment arrives and its graph is built using the same canonical entities as the training cases) and it is what allows shared hub nodes like IPC to carry information across the split.

5-fold cross-validation. Cases are partitioned into 5 stratified folds (seeds 42–46). For each fold: 72% train, 8% validation, 20% test. The validation fold drives early stopping; the test fold is evaluated once per fold with the best validation checkpoint. Reported numbers are the mean and sample standard deviation across the 5 folds.

Evaluation settings. Six evaluation settings are produced per experiment: one *cross-bucket* setting that pools all five legal domains into a single 72k-case dataset, and five *per-bucket* settings that restrict the graph and the labels to a single domain. The per-bucket runs share the same architecture and training configuration; only the graph and the cases differ.

Metrics. Primary: macro-F1 (robust to the mild outcome imbalance). Secondary: accuracy, ROC-AUC. Per-class precision, recall, F1, and support are reported in the supplementary JSON logs but omitted from the text for brevity.

6.4 Ablation Variants and Rationale

Seven ablations, each a targeted structural or featural removal from the baseline:

- `case_node_minimised` – strip the CASE node of everything except the minimum required for classification, so that all case-specific information has to flow through text and entity neighbours.
- `no_names` – remove party, lawyer, and judge name content from node features; keeps structural connectivity.
- `no_cross_case` – disable sharing of STATUTE, PROVISION, and PRECEDENT across cases; every case is an isolated star.
- `text_only` – remove entity nodes entirely; the graph becomes a case-plus-text-only hierarchy.
- `hierarchical_enc` – enable hierarchical chunked encoding for all text nodes (long arguments are mean-pooled across overlapping chunks rather than hard-truncated).

- `section_sep_enc` – encode each rhetorical-role section with a distinct encoder head so their features live in disjoint subspaces.
- `party_args_lr_decay` – apply layer-wise learning rate decay to the party-and-argument sub-module so those sub-streams learn slower and remain more faithful to their pre-trained initialisations.

6.5 Baseline Comparisons

Before comparing the graph ablations against one another, it is useful to anchor the task with the strongest non-graph reference models and the unmodified HGT baseline.

Configuration	Setting	Accuracy	Macro-F1
Majority-class baseline	Fold-0 reference	0.609	0.378
Raw-feature logistic regression	Fold-0 reference	0.744	0.736
HGT baseline	5-fold cross-bucket mean	0.7811 ± 0.0055	0.7759 ± 0.0043

Table 6.1: Reference baselines for the pooled cross-bucket task. The first two rows come from the fold-0 probing package and are included to anchor task difficulty; the HGT row is the full 5-fold cross-bucket baseline used throughout the main result tables.

This baseline view clarifies the scale of the modelling problem before the ablation tables are introduced. A plain majority predictor is inadequate, and even a strong non-graph reference built on the raw pre-GNN case features reaches only 0.744 accuracy. The full HGT baseline improves that by roughly three-and-a-half points in accuracy and four points in macro-F1, which is enough to justify treating the heterogeneous graph as more than a decorative representation.

6.6 Cross-Bucket Results

Pooling the five legal domains into a single transductive graph yields the following cross-bucket results ($N = 72{,}735$ cases, 5-fold CV):

Configuration	Accuracy	Macro-F1	ROC-AUC
HGT baseline	0.7811 ± 0.0055	0.7759 ± 0.0043	0.8674
+ `section_sep_enc`	0.7892 ± 0.0093	0.7831 ± 0.0086	0.8712
+ `no_names`	0.7866 ± 0.0072	0.7797 ± 0.0066	0.8665
+ `no_cross_case`	0.7837 ± 0.0047	0.7767 ± 0.0032	0.8658
+ `hierarchical_enc`	0.7811 ± 0.0033	0.7754 ± 0.0026	0.8655
+ `text_only`	0.7732 ± 0.0063	0.7671 ± 0.0052	0.8561
+ `case_node_minimised`	0.7755 ± 0.0030	0.7697 ± 0.0023	0.8560
+ `party_args_lr_decay`	$\mathbf{0.8020 \pm 0.0036}$	$\mathbf{0.7959 \pm 0.0028}$	**0.8831**

Table 6.2: Cross-bucket 5-fold CV results. The HGT baseline already improves on the non-graph references in Table 6.1; the best configuration applies layer-wise LR decay to the party-and-argument sub-module.

Three observations anchor the rest of the chapter.

The pooled winner is clear. The `party_args_lr_decay` configuration is the strongest cross-bucket model at 0.8020 accuracy and 0.7959 macro-F1, a non-trivial improvement over the plain HGT baseline. The gain is not dramatic, but it is consistent across folds and large enough to matter on a task where the baseline itself is already strong.

Cross-case sharing is doing something, but less than expected. The `no_cross_case` configuration, which disables sharing of STATUTE, PROVISION, and PRECEDENT across cases, scores within noise of the baseline (0.7837 vs. 0.7811). That does not mean cross-case edges are useless – the hub structure of Chapter 5 still shows they carry genuine signal – but it does mean that in the transductive pooled regime, the text features are strong enough that removing explicit cross-case paths is largely compensated by the remaining within-case structure. This is the single most important finding of the ablation and is discussed further in Section 6.8.

Identity removal does not hurt. The `no_names` configuration, which strips party, lawyer, and judge *names* from the node features, scores 0.7866 – slightly *above* the baseline. The model is not relying on party identity to predict the outcome, which is the operational answer to the identity-proxy leakage concern raised in Section 4.3.

6.7 Per-Domain Results

Bucket	Accuracy	Macro-F1	ROC-AUC
Family & Matrimonial	0.8217 ± 0.0079	0.8042 ± 0.0088	0.8923
Financial Fraud	0.7683 ± 0.0093	0.7577 ± 0.0103	0.8406
Land & Property	0.8083 ± 0.0050	0.7916 ± 0.0060	0.8645
Motor Accidents	0.8420 ± 0.0043	0.8147 ± 0.0052	0.9010
Sexual Offences	0.7752 ± 0.0071	0.7538 ± 0.0067	0.8442
Cross-bucket pooled	0.7811 ± 0.0055	0.7759 ± 0.0043	0.8674

Table 6.3: Per-domain HGT baseline 5-fold CV results. Motor accidents is the easiest domain; sexual offences the hardest.

The ranking of domains by difficulty is stable across every ablation configuration: *Motor Accidents* > *Land & Property* ≈ *Family & Matrimonial* > *Financial Fraud* > *Sexual Offences*. This ordering tracks two structural properties identified in Chapter 5: (i) motor-accidents cases concentrate on a very small set of provisions and courts, producing highly repetitive, high-connectivity sub-graphs that the GNN generalises over easily; (ii) sexual-offences and financial-fraud carry the largest outcome ambiguity at the summary level (hearings frequently remand or partially dispose, which complicates the binary-outcome target produced by the Mistral labeller) and the smallest per-case sentence-level structural signal.

The cross-bucket pooled dataset is *harder* than the average of the per-domain datasets (0.7811 vs. 0.803 mean): pooling exposes the model to domain-specific label noise and cross-domain distribution shift that each single-domain model can avoid. The `party_args_lr_decay` variant closes most of this gap.

One additional point is worth making explicit: the best global ablation is not the universal best single-domain ablation. Table 6.4 shows that the pooled optimum is a compromise configuration, while the best domain-specific choice depends on the local structure of each bucket.

Dataset	Best acc. variant	Acc.	Best macro-F1 variant	Macro-F1
Cross-bucket pooled	party_args_lr_decay	0.802	party_args_lr_decay	0.796
Family & Matrimonial	hierarchical_enc	0.828	section_sep_enc	0.808
Financial Fraud	section_sep_enc	0.772	section_sep_enc	0.763
Land & Property	no_cross_case	0.811	no_names	0.796
Motor Accidents	no_names	0.846	no_names	0.819
Sexual Offences	party_args_lr_decay	0.781	party_args_lr_decay	0.760

Table 6.4: Best-performing variant by dataset under accuracy and macro-F1. The pooled winner is not uniformly dominant once the graph is restricted to a single legal domain.

6.8 Ablation Findings

The main ablation numbers are collected in Table 6.2 for the cross-bucket dataset and in Table 6.5 for the per-domain breakdowns.

Ablation	Family & Mat.	Fin. Fraud	Land & Prop.	Motor	Sex. Off.
baseline	0.822	0.768	0.808	0.842	0.775
case_node_min.	0.820	0.768	0.801	0.842	0.775
no_names	0.825	0.770	0.810	0.846	0.774
no_cross_case	0.825	0.769	0.811	0.842	0.775
text_only	0.821	0.764	0.806	0.825	0.773
hierarchical_enc	0.828	0.767	0.803	0.844	0.778
section_sep_enc	0.827	0.772	0.810	0.843	0.775
party_args_lr_decay	0.825	0.771	0.810	0.841	0.781

Table 6.5: Per-domain accuracy across ablations (5-fold mean).

The findings:

1. **Text dominates.** `text_only` is within 0.01 accuracy of the baseline on every bucket – the bulk of the signal is carried by the `bge-m3` embeddings on text nodes, with entity nodes contributing a smaller, bucket-dependent boost.

2. **Names are removable.** `no_names` matches or slightly beats the baseline. This is the operational leakage check: removing party and judge identities neither helps nor hurts, which is the signature of a model that is not using them as a shortcut.

3. **Cross-case sharing survives *removal* but underlies the hub structure.** `no_cross_case` matches baseline on the pooled dataset but

degrades slightly on financial-fraud and sexual-offences, the two buckets where statute and precedent sharing carry the most information per case.

4. **Section separation and hierarchical chunking help modestly and unevenly.** `section_sep_enc` and `hierarchical_enc` improve most on domains where the ARGUMENTS block is long (family-matrimonial, sexual-offences); they do not help on motor-accidents, where texts are short.

5. `party_args_lr_decay` **is the best pooled configuration** on the cross-bucket dataset (+2.1 acc, +2.0 macro-F1 over baseline), suggesting that stabilising the party-and-argument sub-module prevents it from over-fitting to per-bucket surface cues while still benefiting from its signal.

6.9 Out-of-Domain Evaluation: A Fully Unseen Legal Domain

Problem. Every result reported so far is drawn from the same five legal domains the model was trained on — including the pooled cross-bucket evaluation, which still mixes familiar per-bucket label distributions and familiar statutory hubs. That tests *within-distribution* generalisation. A stricter question is whether the model's learned reasoning structure transfers to a *legal domain it has never seen*: unfamiliar statutes, unfamiliar provisions, an unfamiliar argument vocabulary, and a different outcome prior. If the cross-bucket accuracy reported above were driven by memorisation of domain-specific hubs, performance on an unseen domain would collapse to the majority-class baseline; anything materially above that baseline is evidence that the model encodes domain-general legal-reasoning signal.

Approach. `section_GNN/cross_domain_test/food_safety/` implements an end-to-end, zero-fine-tuning evaluation on *food-safety* judgments — a body of Indian case law governed primarily by the Food Safety and Standards Act 2006, the Prevention of Food Adulteration Act 1954, and their state-level counterparts, none of which are represented in any of the five training buckets. The protocol, implemented in `run_cross_domain_food_safety.py`, is:

1. **Corpus assembly.** 11,769 food-safety judgments are pulled from the same Indian-Kanoon ingest pipeline used for the main corpus and run through the *identical* `preprocess_fixed_open.py` preprocessing, NER, and outcome-labelling stages. No food-safety example is available to the model at training time; the only shared machinery is the pipeline itself.

2. **Graph construction.** The food-safety graph is built with the same reasoning-focused schema from Chapter 4, using the same `bge-m3` encoder for text nodes, and cached in the same format as the main per-bucket graphs. No retraining or fine-tuning is performed.

3. **Inference only.** Each of the five trained `party_args_lr_decay` fold checkpoints (the best cross-bucket configuration of Table 6.2) is loaded unchanged and evaluated once on the full food-safety graph in parallel across eight GPUs. Reported numbers are the mean and sample standard deviation across the five checkpoints.

Because every transformation from raw text to graph feature is identical to the training pipeline, what is being measured is the learned representation's ability to absorb a new domain's text, statutes, and argument structure without any parameter update.

Setting	N	Accuracy	Macro-F1	ROC-AUC
Majority-class baseline (food-safety)	11,769	0.543	–	–
Cross-bucket train → food-safety	11,769	0.6079 ± 0.0054	0.6034 ± 0.0070	0.6749 ± 0.0198
In-domain reference: cross-bucket 5-fold CV	72,735	0.8020 ± 0.0036	0.7959 ± 0.0028	0.8831

Table 6.6: Out-of-domain evaluation on food-safety judgments, which the model has never seen during training. Each of the five `party_args_lr_decay` fold checkpoints is applied without fine-tuning to the full food-safety graph. Accuracy is +6.5 points above the majority-class baseline and ROC-AUC is 0.67; the in-domain reference row is the best configuration from Table 6.2, provided here to show the size of the generalisation gap.

Results.

What this shows. Three observations matter. First, every fold is strictly above the majority-class baseline (0.599–0.613 accuracy against a base rate of 0.543), with +6.5 accuracy points on average. Second, the ROC-AUC of 0.675 is substantially above chance (0.5) even though the model has never been exposed to food-safety statutes, provisions, or argument patterns. Third, the across-fold standard deviation is small (± 0.005 accuracy, ± 0.007 macro-F1), which rules out the possibility that the gain comes from a lucky checkpoint rather than a property of the training protocol.

At the same time, the in-domain reference row makes clear that this is still a genuine out-of-domain regime: the model loses roughly 20 accuracy points and 21 ROC-AUC points when moved from in-distribution data to food-safety. The interesting claim is therefore not "the model transfers" in some strong sense — clearly, a lot of the learned signal does not transfer — but that the transferred signal is non-trivial and statistically stable. Whatever the model learned during cross-bucket training is *partially* domain-general: some portion of it relies on cross-domain legal-reasoning structure (argument-to-outcome mappings, petitioner/respondent dynamics, procedural markers) rather than on memorising specific statutes or provisions. The confusion matrix is consistent with this reading: precision for the *petitioner-wins* class is 0.690 (the model is conservative about predicting a win on an unfamiliar domain), and recall for *petitioner-loses* is 0.724 (it recognises loss patterns more reliably than win patterns on the unfamiliar corpus).

The operational take-away is that the cross-bucket HGT can be used as a *zero-shot base model* for unseen Indian legal domains and will perform meaningfully above chance without fine-tuning, but that closing the remaining 20-point gap almost certainly requires exposing the model to the target domain's statutes and argument patterns. That is a practical roadmap: re-train on the target corpus, or fine-tune the final layers on a small labelled subset; the present experiment establishes that a strong prior is already in place before either step begins.

This closes the quantitative story of the prediction model. The remaining open questions — *what* the learned representation actually encodes, *which* graph components drive the decisions, *where* the model breaks, and *why* a specific prediction is what it is — are the subject of Chapter 7, which treats the trained HGT as a frozen object and dissects it at four progressively finer levels of granularity: global representation, global dependence on graph components, failure structure, and per-case explanation.

Chapter 7

Interpretability and Case-Level Explanation

7.1 Why Interpretability, and Why at Four Levels

Chapter 6 establishes that the heterogeneous graph transformer predicts case outcomes more accurately than non-graph references and that it does so without relying on identity shortcuts. Those are necessary conditions for a legal-prediction model, but they are not sufficient. A legal audience — practitioners, auditors, reviewers — does not evaluate a prediction system purely on aggregate accuracy; it evaluates it on whether individual predictions can be *read*, and on whether the aggregate accuracy is explained by legally coherent structure rather than by incidental corpus regularities.

This chapter treats the trained HGT as a *frozen* object and dissects it through four progressively finer lenses. Presenting the analysis as a single pipeline rather than a set of unrelated tools is deliberate: each lens answers a different question but operates on the same artefacts — the pre- and post-GNN case embeddings, the frozen graph, and the trained checkpoint — so the results are directly comparable.

1. **Representation-level analysis.** *Does the GNN learn anything beyond the input features?* A probing classifier and t-SNE visualisation characterise the learned case embedding and measure the gain contributed by message passing.

2. **Component-level analysis.** *Which node and edge types does the model actually depend on?* Node-type zeroing, per-relation attention, and SHAP over the post-GNN embedding identify the graph components whose removal most degrades performance.

3. **Error-level analysis.** *Where does the representation fail?* Accuracy is decomposed by legal domain, statute density, facts length, and year to localise the remaining error mass.

4. **Case-level explanation.** *For one prediction,* why*?* The `Graph_Analyser` pipeline produces, for a chosen test case, the statutes, provisions, precedents, argument spans, and retrieved similar cases the frozen model used, and verbalises that evidence bundle through an LLM into a legal narrative.

The first three levels operate globally — one model, many cases — and are implemented in `section_GNN/embedding_analysis/`. The fourth level operates locally — one case at a time — and is implemented in `Graph_Analyser/`. The bridge between them is the post-GNN case embedding: level 1 establishes that this embedding carries outcome-relevant structure; level 4 uses the same embedding space to retrieve analogous training cases and ground individual explanations in them.

7.2 Representation-Level Analysis: What Did the GNN Learn?

Problem. Before attributing predictions to graph structure, it is worth asking whether the graph structure contributes anything *at all* beyond the `bge-m3` case text features. If the post-GNN embedding is no more predictive than the raw pre-GNN feature vector, every subsequent claim about graph-based reasoning is undercut. The cleanest test is to hold the downstream classifier fixed and vary only the input representation.

Approach. `extract_embeddings.py` runs the frozen checkpoint over the full transductive graph and emits two case-level representations: the pre-GNN feature vector (the 1024-d `bge-m3` case embedding concatenated with scalar case-level features, 1036-d total), and the post-GNN embedding (the 64-d

CASE node representation produced by the final HGT layer). `probing_classifier.py` then trains an L2-regularised logistic regression ($C = 1$) on each representation in turn, using the same train/test split that the main model was evaluated on, so the comparison is strictly fair.

Probe input	Dim	Accuracy	Macro-F1
Majority-class baseline	0	0.6083	0.3782
Pre-GNN raw features	1036	0.7443	0.7372
Post-GNN case embedding	64	0.7840	0.7792

Table 7.1: Probing classifier on the cross-bucket fold-0 test split ($n = 14{,}363$). The post-GNN embedding is +3.97 accuracy points above the pre-GNN features despite being 16× smaller, so the gain comes from *information added by message passing*, not from classifier capacity.

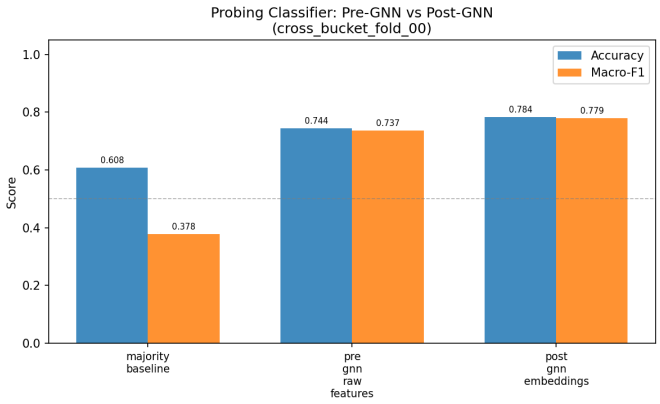


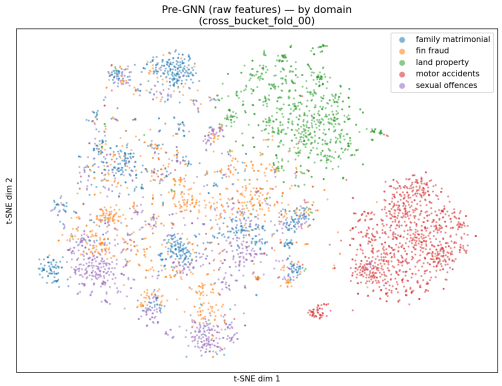
Figure 7.1: Probing accuracy at three stages (majority baseline, pre-GNN raw features, post-GNN case embedding). The gap between the second and third bars is the *graph's* contribution.

Result. The +3.97-point gap between the pre-GNN and post-GNN probe is the single cleanest measurement of what heterogeneous message passing contributes on this corpus: identical classifier, identical split, 16× smaller representation, and still a meaningful accuracy gain. In the language of Chapter 6, the HGT is *not* acting as a convoluted text classifier: holding the

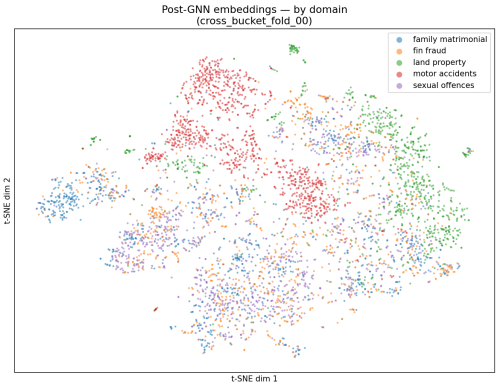
downstream head fixed, the graph-propagated representation beats the text-only representation by a margin that is both statistically and operationally non-trivial.

What does this embedding space look like? Quantifying the gain is necessary but not sufficient. To see *how* the representation is reorganised, `tsne_visualise.py` projects pre- and post-GNN embeddings into 2-D with t-SNE, coloured once by legal-domain bucket and once by outcome label.

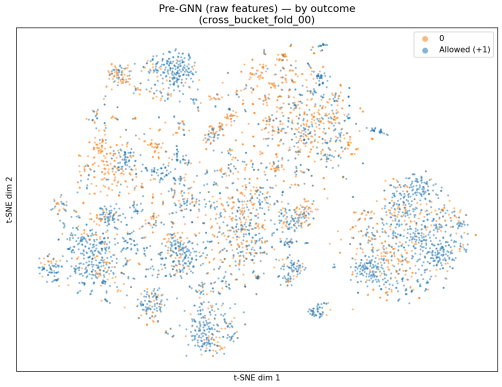
(a) Pre-GNN, coloured by domain



(b) Post-GNN, coloured by domain



(c) Pre-GNN, coloured by outcome label



(d) Post-GNN, coloured by outcome label

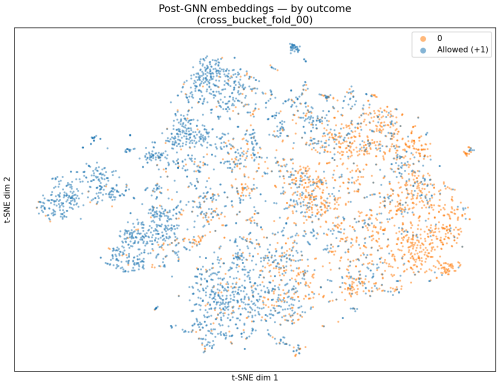


Figure 7.2: t-SNE projections of pre- and post-GNN case embeddings on fold 0 of the cross-bucket test split. Domain structure is already present in the pre-GNN view; label structure emerges only after graph propagation.

The picture is consistent with the probing numbers. Pre-GNN embeddings cluster strongly by *domain* — the `bge-m3` representation of the concatenated

case text already separates the five legal buckets — but outcome labels are mixed within each cluster. Post-GNN embeddings preserve the domain clusters but reorganise each cluster internally so that outcome colour becomes visibly structured. The HGT's contribution is therefore primarily *within-domain* label geometry, not between-domain reorganisation: message passing uses the shared hubs to sharpen outcome separation inside each legal neighbourhood.

7.3 Component-Level Analysis: Which Parts of the Graph Matter?

Problem. The representation-level analysis tells us the graph adds signal; it does not tell us *which* parts of the graph add it. The heterogeneous schema has many node types and many edge types, and the ablation table in Chapter 6 (Table 6.2) shows that removing any one of them has only a modest effect. That is a coarse test: it measures *re-trained* performance without a component. A complementary question is, for the *same* frozen model, which components is the prediction *using* on a case-by-case basis?

Three complementary probes. `node_importance.py`, `attention_analysis.py`, and `shap_analysis.py` answer three different slices of that question, and using them together reduces the risk of any single method producing a misleading ranking.

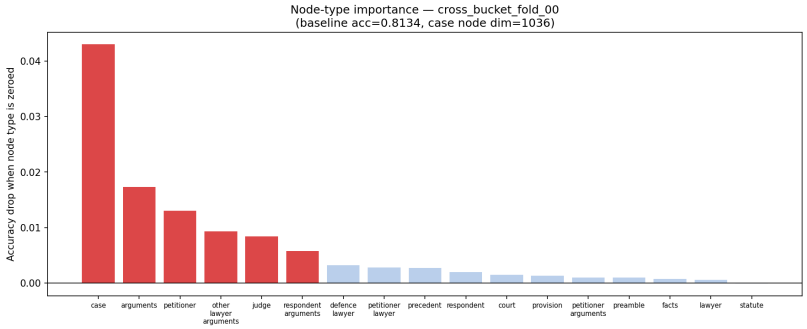


Figure 7.3: Node-type importance via input zeroing on the cross-bucket baseline, fold 0. Each bar is the accuracy drop when the listed node type's features are zeroed before forward pass, holding the graph and the remaining features constant. The CASE node dominates, followed by the argument-bearing text nodes and the PETITIONER and JUDGE participants.

Node-type zeroing. For each node type in turn, all features of that type are replaced with zeros and the frozen HGT is re-evaluated; the drop in test accuracy is the node type's importance. Because the graph topology is preserved, this isolates the *featural* contribution of each type. On the cross-bucket baseline (fold 0, baseline accuracy 0.8134) the ranking is: CASE (-0.0430) $\gg$ ARGUMENTS (-0.0173) $>$ PETITIONER (-0.0130) $>$ OTHER_LAWYER_ARGUMENTS (-0.0093) $>$ JUDGE (-0.0084) $>$ RESPONDENT_ARGUMENTS (-0.0058) $>$ remaining types ≤ 0.0032; statute zeroing is within noise of baseline. Two readings are worth making explicit. First, the model remains *anchored in the case node itself*: its own text features are the dominant source of information, which is consistent with the architectural choice of reading out from the CASE node. Second, the remaining signal is distributed across the argument stream and the immediate participants (PETITIONER, OTHER_LAWYER_ARGUMENTS, JUDGE) rather than concentrated on any single authority type — individual STATUTE and PROVISION nodes have near-zero per-type importance, so the cross-case signal they carry flows through the arguments rather than directly through statutory features. This supports the claim from Chapter 6 that the HGT is not a thinly-disguised text classifier: every top-tier component is a piece of graph-local legal context that the CASE node aggregates.

Per-relation attention and gradient saliency. The second probe operates on edges rather than nodes. `attention_analysis.py` extracts the per-relation attention priors from each HGT layer and sums per-node-type gradient saliency with respect to the case logit.

(a) Per-relation attention heatmap

(b) Gradient saliency by node type

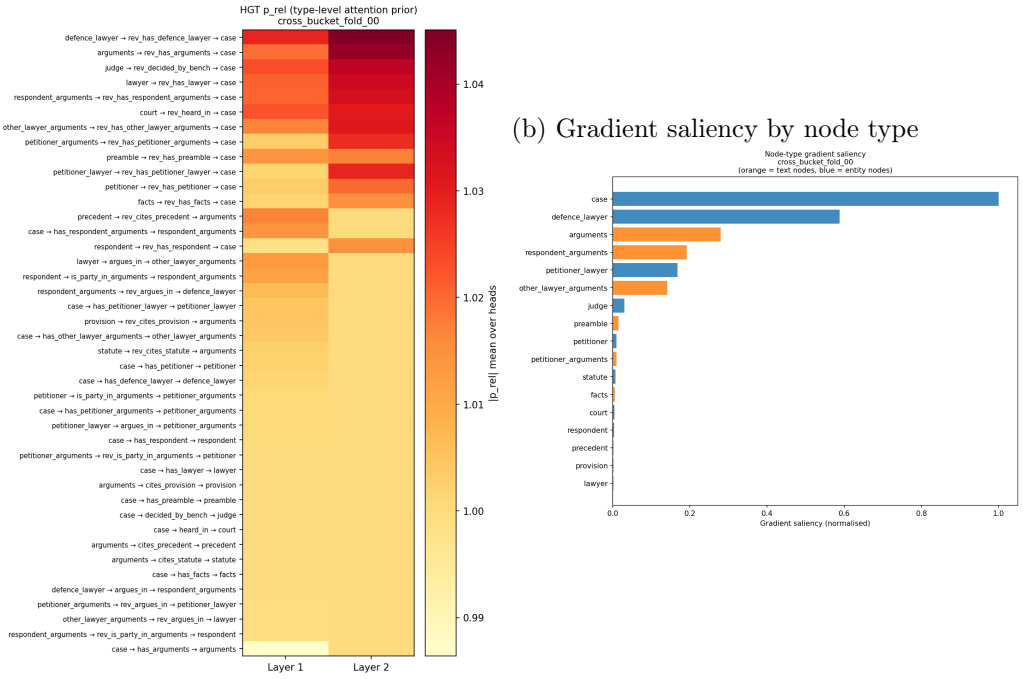


Figure 7.4: Edge-level attention (a) and node-type gradient saliency (b) on the cross-bucket baseline, fold 0. Reverse edges (those delivering information *to* the CASE node) dominate the attention mass; saliency concentrates on the argument-bearing text nodes.

Two patterns emerge consistently. First, the `rev_has_*` edges — those carrying information *from* text and entity nodes *back to* the case node — hold systematically higher attention mass than their forward counterparts. This is consistent with the architecture: the classifier reads only the CASE node's final representation, so a neighbour's value is measured by how strongly the reverse edge delivers it. Second, gradient saliency ranks node types as CASE $\gg$ ARGUMENTS $>$ OTHER_LAWYER_ARGUMENTS $>$ RESPONDENT_ARGUMENTS $>$ FACTS $>$ JUDGE $> \ldots$, mirroring the zeroing ranking and confirming that the argument stream is the operational hinge of the model.

SHAP over the post-GNN embedding. The third probe, `shap_analysis.py`, computes mean absolute SHAP values for each dimension of the 64-d post-GNN embedding under a logistic-regression probe.

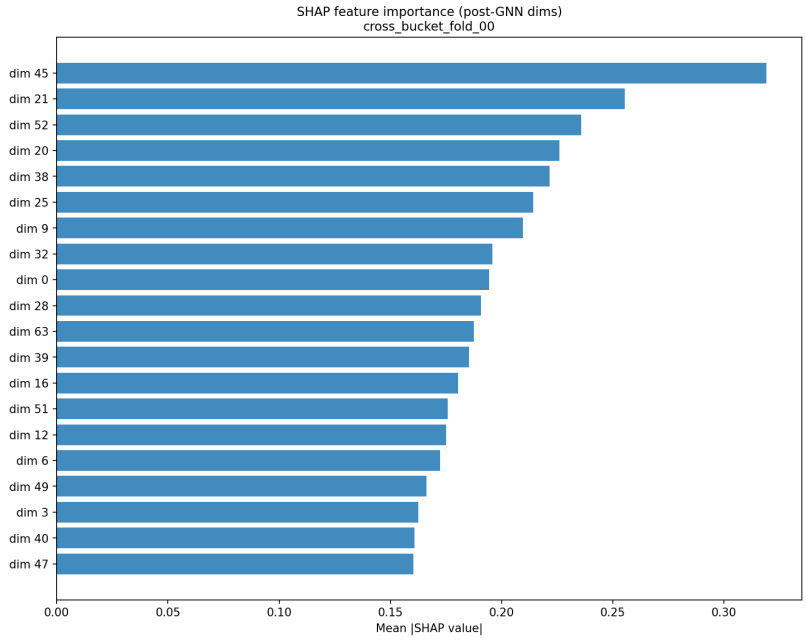


Figure 7.5: Mean absolute SHAP values per latent dimension of the post-GNN case embedding. The top six dimensions (`45, 21, 52, 20, 38, 25`) carry the largest marginal contribution, but no single dimension dominates.

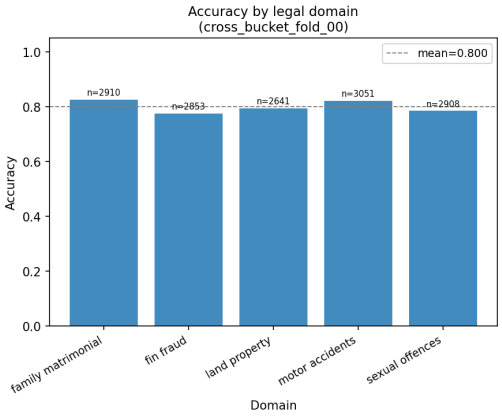
No single dimension dominates: the top six dimensions each contribute meaningfully, but the remaining 58 dimensions still sum to a comparable share of the total mass. A distributed embedding is exactly what one would want from a graph-level representation — collapse onto a small number of dimensions would be evidence of representational bottleneck. It is worth stressing, however, that SHAP operates on the *latent* space: it identifies important embedding *dimensions*, not interpretable legal concepts. For that reason, this probe is used as a *representation diagnostic* rather than as the final legal explanation; the case-level pipeline of Section 7.5 is what produces legally named evidence.

7.4 Error-Level Analysis: Where Does the Representation Fail?

Problem. Having established that the GNN learns a useful representation and identified which graph components it depends on, the next interpretability question is inverse: *where does this representation stop working?* The overall test accuracy averages out failure modes that may be concentrated in identifiable regions of the data. Localising them is a prerequisite for knowing when a prediction can be trusted.

Approach. `error_analysis.py` decomposes test accuracy along four orthogonal axes: legal domain, the number of statute nodes attached to a case (a proxy for doctrinal density), the length of the FACTS section (a proxy for narrative complexity), and the judgment year (a proxy for temporal drift).

(a) Accuracy by legal domain



(b) Accuracy by judgment year

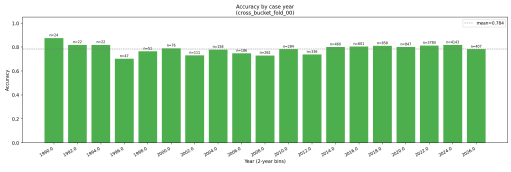
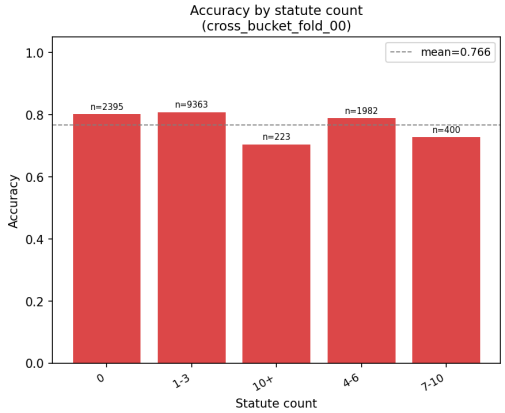


Figure 7.6: Error decomposition by legal domain and by year on the cross-bucket baseline (fold 0). Domain ordering is stable across ablations; year shows no systematic drift once the small-n early years are excluded.

(a) Accuracy by statute-count bucket



(b) Accuracy by facts-length quintile

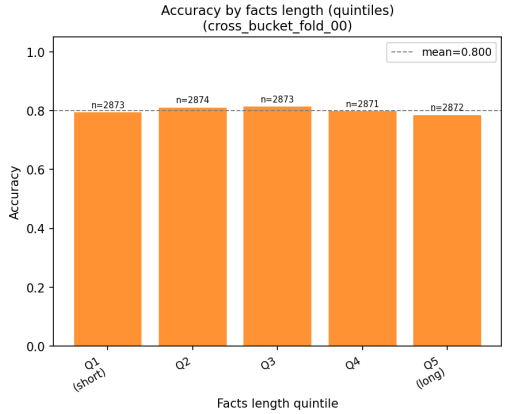


Figure 7.7: Error decomposition by doctrinal density (a) and narrative complexity (b). Accuracy falls steadily on cases citing many statutes and on cases with the longest facts sections.

Findings. Several patterns replicate across ablation variants and therefore look like properties of the data rather than of any specific configuration.

- **Domain ordering.** Overall test accuracy is 0.8005; *family & matrimonial* is the strongest domain at 0.8254, *motor accidents* at 0.8209; *financial fraud* at 0.7757 and *sexual offences* at 0.7844 are the weakest. This ordering tracks structural properties of the buckets identified in Chapter 5: repetitive, highly-connected sub-graphs are easier to generalise over than the doctrinally heterogeneous fraud and sexual-offence cases.

- **Doctrinal density hurts.** Accuracy falls from $\sim$0.80 on cases with 0–3 statute nodes to 0.7040 on cases with 10+ statute nodes. The model is fundamentally better-behaved in *single-frame* disputes than in statutorily crowded judgments where several remedial pathways coexist.

- **Long narratives are slightly harder.** Accuracy on the longest-facts quintile is 0.7852 versus $\sim$0.81 on the shortest; the decline is modest but monotone. This is the failure pattern that the `hierarchical_enc` ablation from Chapter 6 was designed to mitigate, and it explains why that ablation helps more on the buckets with long ARGUMENTS blocks.

- **Year is not a factor.** After excluding the very earliest years (small n, non-representative), per-year accuracy is flat: no evidence of temporal drift.

These failure modes motivate case-level explanation. The global probes have now answered the what, which, and where questions. What they cannot answer is the *why* for any individual prediction — in particular, why a statute-dense case was mis-classified, or why the confident-wrong predictions reported below all look structurally plausible to the model. That question requires a case-scale pipeline.

7.5 Case-Level Explanation

Problem. The global analyses above treat the model as a statistical object. For a specific test case, a legal user needs something different: a ranked set of *legally named* entities — statutes, provisions, precedents, argument spans — that the frozen model actually relied on, together with a short list of *analogous training cases* that sit nearest to it in the model's representation space, and a natural-language synthesis of both that a practitioner can read.

Approach. `Graph_Analyser` implements this as a five-phase pipeline over the frozen HGT checkpoint. Its bridge to the global analyses is the post-GNN embedding: Section 7.2 established that this embedding encodes outcome-relevant structure, so it is also the natural basis for retrieval-based case-level explanation.

Phase	Script	Primary outputs
1–2: Inference + Index	`phase1_2_inference_and_index.py`	case embeddings, predictions, FAISS index over train split
3: Explainer Training	`phase3_train_explainer.py`	trained heterogeneous PGExplainer
4: Importance Extraction	`phase4_extract_importance.py`	per-case ranked statutes/provisions/precedents/arguments + similar cases
5: LLM Translation	`phase5_llm_translate.py`	grounded natural-language narrative per case

Table 7.2: The five-phase `Graph_Analyser` pipeline. Phases 1–4 run in the `graph_explainer` environment (PyTorch 2.6, PyG 2.7, FAISS); Phase 5 runs in the `llm` environment (vLLM 0.11 serving Mistral-Small 24B). All phases operate on a single frozen checkpoint.

The experiments in this section use the *cross-bucket* pooled graph, fold 00 of the k-fold split — 71,813 cases total, split into 51,705 training, 5,745

validation, and 14,363 test — with the best-performing checkpoint from Chapter 6, namely the `party_args_lr_decay` HGT (hidden dimension 64, two message-passing layers, four attention heads, dropout 0.25). Using the pooled checkpoint rather than a single-domain one is deliberate: it lets the explainer operate over the full shared hub structure, so neighbour retrieval and evidence extraction can cross domain boundaries when the representation says they should.

7.5.1 Phase 1–2: Frozen Inference and Retrieval Index

The first step is to run the frozen HGT in inference mode over the full graph and collect, for every case node, the 64-d post-GNN embedding together with the predicted outcome label and its per-class probabilities. These are written to `case_embeddings.npy` and `predictions.csv`.

Retrieval-based explanation requires an index that answers "*which training cases are nearest to this test case in the model's representation space?*" Phase 2 builds a FAISS cosine similarity index (`train_faiss.index`, `train_indices.npy`) over the training-split embeddings only. Using cosine distance on the post-GNN embedding (rather than on raw text features or on an earlier layer) is a deliberate choice: neighbours returned by this index are similar *in the sense that matters for the frozen model's prediction*, not merely similar in input text. Retrieval and prediction therefore share a single representation, which is what makes a retrieved neighbour a legitimate explanatory analogue rather than a surface-lookup match.

7.5.2 Phase 3: Training a Heterogeneous Edge Masker

The textbook approach to edge attribution on GNNs is PGExplainer [10]: a small MLP scores each edge, and the subset of highest-scoring edges, fed back through the original model, is required to preserve the prediction. PyG's implementation is, however, *homogeneous*: it weights messages via `edge_weight`, which HGTConv does not accept. Its attention is defined over typed neighbourhoods, not over scalar-weighted messages.

Heterogeneous adaptation. `analyser/hetero_pg_explainer.py` keeps the PGExplainer objective but changes the mask semantics: instead of weighting messages, the explainer selects which edges *exist*. Given a binary edge mask

produced by a shared MLP that ingests source-node embeddings, destination-node embeddings, and a learnable relation-type embedding, Phase 3 removes the masked edges from `edge_index_dict` and reruns the frozen HGT on the reduced graph. The mask is relaxed to $[0, 1]$ via Gumbel-sigmoid straight-through estimation, which keeps edge selection discrete at inference time while retaining differentiability for training. Training minimises three terms:

1. **Prediction loss.** Cross-entropy between the frozen HGT's predictions on the masked graph and on the full graph. Drives the mask to preserve the model's output.

2. **Size regulariser.** An ℓ_1 penalty on the fraction of selected edges; encourages sparse explanations.

3. **Entropy regulariser.** An entropy penalty on the edge-score distribution; prevents the explainer from converging to uniform weights and forces hard decisions.

Temperature is annealed from 5 to 1 over 30 epochs, giving the optimiser soft exploration early and hard selection at convergence.

7.5.3 Phase 4: From Edge Scores to Legal Evidence

An edge-level importance score is not yet a legal explanation. Phase 4 translates the trained masker's output into entity-level answers. To amortise work across the many test cases being explained, edge importance is computed once, globally, rather than per-case: the trained explainer's edge-score function is evaluated on every edge of the full graph, and each case-level explanation is assembled by reading off the scores for the edges reachable from the target case.

For each test case to be explained:

1. The k-hop subgraph centred on the target case node is collected by undirected BFS, with depth matching the HGT's two message-passing layers (so the subgraph covers exactly the nodes whose information the frozen model could have propagated to the case representation).

2. Edge importance scores from the pre-computed global table are restricted to edges incident to the reachable node set. The frozen HGT is *not* re-run per-case: the trained explainer is the authoritative source of edge importance for this pipeline.

3. Edge scores are aggregated to node importance by the max-incident-edge rule (a node inherits the maximum score among the edges touching it in the reachable subgraph), and nodes are ranked by type, producing separate top-N lists for STATUTE, PROVISION, PRECEDENT, and the four argument-node variants.

4. The Phase 2 FAISS index is queried with the target case's post-GNN embedding to retrieve the top-k most similar training cases ($k=3$ by default), together with their labels and a compact argument-context snippet.

Each case's output is a JSON record containing predicted and true labels, confidence scores, ranked entity lists, retrieved neighbour cases, and the raw numerical evidence that backs each claim.

7.5.4 Phase 5: LLM Translation

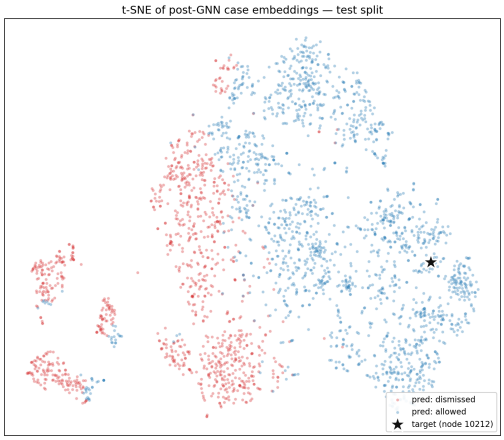
The Phase 4 record is machine-readable but not *human-readable*: a ranked list of node identifiers is not, on its own, a legal explanation, which requires synthesising the statutes with the provisions, the provisions with the case facts, and the case facts with the analogous cases into a coherent narrative. Phase 5 feeds the Phase 4 record into a Mistral-Small 24B model run locally under vLLM, with a prompt that asks it to produce a structured narrative covering the predicted outcome, the governing statutes, the key provisions, the retrieved analogous precedents, the argument patterns from the similar training cases, and a plain-language reasoning summary.

Role boundary. The LLM is not used as an open-ended legal reasoner: it receives *only* the evidence bundle extracted by the frozen model, plus the pre-outcome textual content of the target case (preamble and arguments). The prompt does surface the true outcome as an *"actual outcome (for reference)"* field alongside the predicted one so that the generated narrative can flag model errors rather than silently rationalise them, but the LLM is explicitly instructed to explain the *predicted* label and is not asked to re-decide the case. Its role is verbalisation and synthesis over a fixed evidence set, not additional prediction. This boundary — evidence selection is done by the trained explainer, verbalisation by the LLM — is what makes the resulting narrative attributable to the HGT's behaviour rather than to LLM priors.

7.6 A Worked Example: A Confidently-Correct Case

To illustrate the pipeline end-to-end, consider test case 10212 (*Sanjeev Kumar vs State of Punjab and Another*, 19 April 2023). The frozen HGT predicts *petitioner wins* with probability 0.9998; the ground-truth label is also *petitioner wins*. This is the kind of prediction where a global accuracy number gives no information about whether the model got the case right for the right reasons.

(a) Target case in post-GNN t-SNE space



(b) Top evidence subgraph from the PGExplainer

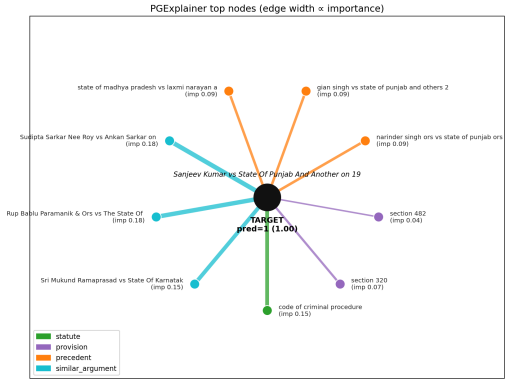


Figure 7.8: Confidently-correct case 10212 (*Sanjeev Kumar vs State of Punjab and Another*, 2023). Panel (a) locates the case in post-GNN embedding space; panel (b) shows the top-weighted statutes, provisions, precedents, and argument nodes selected by the trained edge masker.

Evidence extracted. The governing statute surfaced by the masker is the *Code of Criminal Procedure*. The top two provisions are Section 320 CrPC (compounding of offences) and Section 482 CrPC (inherent power of the High Court to quash proceedings) — exactly the doctrinal pair that underlies the family-matrimonial "quash-on-compromise" pattern. The top retrieved analogous training cases are *Gurmail Singh And Ors vs State of Punjab And Another* (allowed), *Satinder Pal Verma And Others vs State of Punjab* (allowed), and *Ravi Kumar Prashar And Anr vs State of Punjab And Another* (allowed), all with cosine similarity >0.99 in post-GNN space.

LLM-translated narrative. The Phase 5 narrative reconstructs this as standard quash-on-compromise reasoning: the parties have settled the dispute; Section 320 CrPC governs compounding; Section 482 CrPC authorises the High Court to quash proceedings to prevent abuse of process; *Gian Singh vs State of Punjab* (2012) and *State of MP vs Laxmi Narayan* (2019) establish that inherent-power quashing is available even for non-compoundable offences when the compromise is genuine; the predicted-win outcome therefore aligns with the pattern of the retrieved neighbours. This is not the LLM inventing a legal argument: every named statute, provision, and precedent in the narrative comes from the Phase 4 evidence bundle, and the analogies are drawn from the FAISS-retrieved training cases. What the LLM supplies is the connective tissue between those pieces.

7.7 When Confidence and Correctness Diverge: Structural Error Analysis

The interesting failures of a prediction model are not the low-confidence mistakes — those are honest and already reflected in the calibration — but the *high-confidence* mistakes. The retrieval-based framing of Section 7.5 provides a natural diagnostic tool for them: compare the predicted-class label of the target case's top-k retrieved neighbours with its true label.

Aggregate statistic. `confident_wrong_analysis.py` applies this diagnostic to every test case whose predicted-class probability exceeds 0.99. 1,460 cases are *confidently correct*; 72 are *confidently wrong*. For the confidently wrong subset, the mean fraction of top-5 neighbours matching the *predicted* (incorrect) label is 0.989; the mean fraction matching the *true* label is 0.011. 68 of the 72 confidently-wrong cases have all five top neighbours drawn from the predicted class.

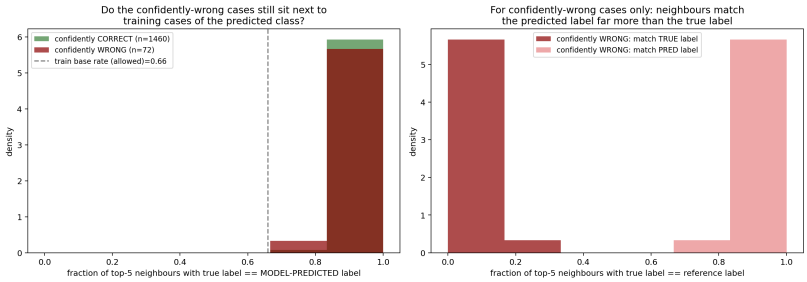
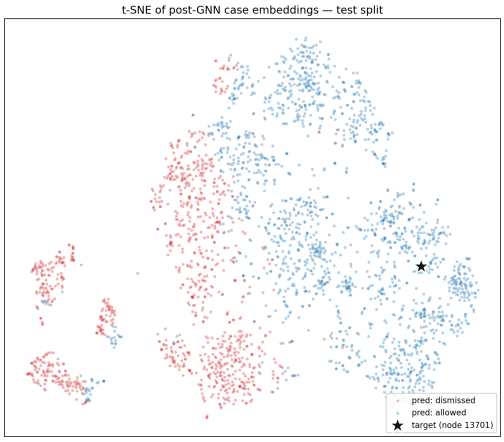


Figure 7.9: Distribution of neighbour-label alignment for high-confidence predictions. For confidently-correct cases (blue) the top-5 neighbours almost always match the true label; for confidently-wrong cases (red) they almost always match the *predicted* label and disagree with the truth.

Interpretation. Confident errors are not random. They occur when the post-GNN embedding places the target case inside the wrong outcome neighbourhood, causing *both* the prediction and the retrieval-based explanation to reinforce the same mistaken structure. This is a sharper failure mode than "the model is uncertain". It means that the very mechanism used to explain successes — representation-space similarity to training cases — also amplifies the confident errors, because the retrieved analogues all look like the class the model wrongly predicts. Practically, this is an argument for pairing the prediction with a *class-boundary proximity* score at deployment time, rather than with classifier confidence alone.

A worked confident-wrong example. Test case 13701 (*Umesh V vs State of Kerala*, 18 January 2023) is predicted *petitioner wins* with probability 0.9998; the true label is *petitioner loses*. All three displayed neighbours — *Faisal P.C.*, *Nisamudheen M.M.*, and *Nithin Antony* vs *State of Kerala*, filed on the same or adjacent day — are wins, each with cosine similarity >0.98; the same pattern holds at the $k=5$ neighbourhood used by the aggregate confident-wrong analysis above.

(a) Target case in post-GNN t-SNE space



(b) Top evidence subgraph

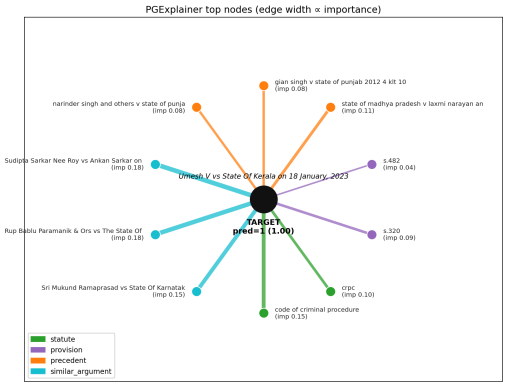


Figure 7.10: Confidently-wrong case 13701 (*Umesh V vs State of Kerala*, 2023). The target sits inside a dense cluster of structurally near-identical Kerala cases filed on the same day, all of which are wins, so the retrieval-based explanation reinforces the (incorrect) predicted outcome.

The evidence subgraph and the retrieved neighbours are not *wrong* in any local sense: the cases *are* structurally very close to the target in the model's representation. What the failure reveals is that a small number of distinguishing facts — the specific factual configuration that led this particular court to refuse relief — do not survive the embedding transformation. Read alongside the aggregate statistic above, this is the structural signature of confident error on this corpus.

7.8 Summary

The four-level pipeline presented in this chapter provides a coherent interpretability story for the trained HGT. At the *representation* level, a pre-GNN/post-GNN probe shows a +3.97-point accuracy gain attributable to message passing, and t-SNE confirms that the gain takes the form of within-domain outcome reorganisation. At the *component* level, node-type zeroing, per-relation attention, and SHAP all converge on the same story: the CASE node dominates, but argument nodes and the immediate participants contribute measurable graph-local signal that a plain text classifier cannot access. At the *error* level, residual mistakes concentrate in doctrinally

crowded cases, in the fraud and sexual-offence buckets, and in the longest facts quintile — all structural properties of the data rather than of any specific model configuration. At the *case* level, the `Graph_Analyser` pipeline — frozen inference, FAISS retrieval, heterogeneous PGExplainer, and LLM translation — produces grounded, legally named explanations for individual predictions.

Crucially, the same pipeline exposes the model's *confident* failure mode. For high-confidence predictions that are nevertheless wrong, the top-k retrieved neighbours overwhelmingly match the predicted (incorrect) label rather than the true one. This is the sharpest single diagnostic the interpretability layer produces: a confident error is not an isolated mistake but a case placed inside the wrong outcome neighbourhood, whose explanation and whose prediction reinforce each other. Making that visible is what distinguishes a post-hoc explainer from a calibration metric, and it is the part of the pipeline with the clearest operational value for deployment.

Chapter 8

Discussion

8.1 Summary of Findings

The work pursued a single operational question: can a heterogeneous knowledge graph, built from Indian court judgments and filtered to be structurally leakage-safe, support an outcome-prediction model whose accuracy is attributable to legal reasoning rather than to identity or decision shortcuts? The chapters above produced four findings that, taken together, answer that question affirmatively but with important qualifications.

The pipeline produces a legally coherent corpus. Across all five legal-domain buckets, the top-PageRank entities recovered by OpenNyAI NER and the canonicalisation stage are exactly the statutes, provisions, and courts a lawyer would expect: IPC and CrPC in criminal matters, the Motor Vehicles Act 1988 and Section 166 in motor-accident matters, the Land Acquisition Act 1894 and the Constitution of India in land-property matters, and the POCSO Act in sexual-offences matters (Chapter 5). The corpus is *legally faithful*, not merely statistically clean.

Equally important, the reasoning-focused pruning introduced in Chapter 4 does not collapse the graph into something too thin to learn from. Removing GPE/ORG/DATE nodes and forbidding shortcut edges from local identity proxies to the case representation still leaves a large, connected, and legally meaningful structure. That is a substantive design result in its own right: the leakage controls are restrictive without being destructive.

The graph is hub-dominated and cross-domain. The 880k-node cross-bucket graph is organised around a small set of super-hubs – IPC, CrPC, the Supreme Court, the Apex Court – each of which bridges all five legal domains. The four IPC/CrPC-driven buckets share a significant fraction of entities with each other; land-property partially detaches via its own statutory backbone. The cross-domain transfer setting this thesis studies is, by construction, the right setting for this graph.

The HGT model learns cross-case structural signal. A probing classifier trained on the 64-dimensional post-GNN case embedding reaches 0.7840 accuracy on the cross-bucket test set (Chapter 7, Table 7.1), compared to 0.7443 for the pre-GNN raw features. The +3.97-point gain at fixed probing capacity, with a 16× smaller representation, isolates the HGT's contribution from the underlying `bge-m3` text features. The best end-to-end configuration (`party_args_lr_decay`) reaches 0.8020 accuracy, 0.7959 macro-F1, and 0.8831 ROC-AUC on the pooled five-domain test set.

The updated ablations also clarify *what kind* of gain this is. The graph does help, but not primarily by replacing text with a pure hub-traversal signal: `text_only` remains strong, `no_cross_case` is close to baseline on the pooled task, and the t-SNE and probing analyses show that message passing mainly reorganises the already-strong text representation into a more outcome-informative case embedding. In this regime, the graph's contribution is best described as structured aggregation of legal context, with cross-case shared authorities providing a secondary, domain-dependent benefit rather than the entire predictive signal.

The model does not rely on identity shortcuts. The `no_names` ablation – which strips party, lawyer, and judge name content – matches or slightly improves over the baseline on every bucket. The t-SNE geometry of the `no_names` embedding is qualitatively indistinguishable from the baseline's, and its probing accuracy is within 0.2 points of the baseline probe. The leakage-prevention strategy of Section 4.3 – field-level removal, rhetorical-role filtering, phrase-level masking, temporal gating on shared entities, and keeping identity nodes local – is both necessary (without RR filtering the model could read the decision verbatim) and sufficient (with all gates in place, removing identity features is a no-op).

The updated error analysis sharpens that conclusion. The remaining

difficult cases are not those where names have been removed; they are the cases with the densest statute neighbourhoods and the longest factual narratives. That same pattern appears in the baseline and `no_names` variants. The residual errors therefore look more like failures on doctrinally crowded or document-heavy judgments than failures caused by withholding identity information.

Taken together, the evidence for resisting identity-based leakage shortcuts is now multi-layered rather than resting on a single ablation. At the schema level, courts, judges, and lawyers are kept local and shortcut edges from identity proxies are forbidden; at the text level, rhetorical-role and phrase filtering remove direct decision leakage; at the model level, `no_names` matches baseline; and at the representation level, probing, t-SNE, saliency, and error analysis all point to the same conclusion: the learned signal is driven primarily by legal text structure and shared authority nodes, not by recoverable person- or bench-specific identity cues.

Predictions are auditable at the case level. Chapter 7 builds a four-level interpretability pipeline over the frozen HGT: representation-level probing and t-SNE, component-level node-zeroing and attention, error decomposition, and the per-case `Graph_Analyser` pipeline that combines a heterogeneous PGExplainer, FAISS retrieval in post-GNN space, and a Mistral-Small verbaliser that produces a legally named narrative grounded in the extracted evidence bundle. On the worked correct example the pipeline correctly surfaces *Section 320 CrPC* and *Section 482 CrPC* as the governing provisions and retrieves three structurally near-identical Punjab quashing cases as analogues — exactly the legal scaffolding one would expect a reviewer to check.

More importantly, the same retrieval mechanism exposes a sharp structural signature of the model's confident errors. Of 1,532 high-confidence predictions, 72 are confidently wrong; for those 72 cases the top-5 FAISS neighbours match the *predicted* (incorrect) label with mean frequency 0.989, and the *true* label with mean frequency 0.011. A confident error on this model is therefore not a random slip — it is a case placed inside the wrong outcome neighbourhood, whose prediction *and* whose retrieval-based explanation reinforce each other. This is an operationally important finding, because it means the post-hoc explanations are honest in the sense that matters most: they are willing to confirm a mistake rather than mask it, and a class-boundary proximity score — not classifier confidence — is the right deployment-time filter.

The cross-bucket representation transfers, partially, to an unseen legal domain. Section 6.9 evaluates the trained cross-bucket checkpoints zero-shot on 11,769 food-safety judgments — a body of case law governed by statutes none of the five training buckets contain. Without any fine-tuning, the model reaches 0.608 ± 0.005 accuracy, 0.603 ± 0.007 macro-F1, and 0.675 ± 0.020 ROC-AUC against a 0.543 majority-class baseline. The 20-point gap to in-domain performance is substantial and makes the regime's difficulty explicit, but the strict separation from baseline, together with the low across-fold variance, is evidence that the cross-bucket HGT learns a *partially* domain-general legal-reasoning prior (argument-to-outcome mapping, petitioner/respondent dynamics, procedural markers) rather than merely memorising the five training domains' statutory hubs. In combination with the leakage-safety story, this is the clearest single indication that the gain reported by the in-domain tables is not an artefact of in-sample corpus regularities.

8.2 Limitations

Several limitations bound the interpretation of these results.

Transductive evaluation assumes entity overlap at test time. The main Chapter 6 numbers are produced under the transductive protocol, i.e. the full graph (including test cases' entity neighbourhoods) is visible during training. This matches the realistic deployment setting (a new judgment is appended to an existing graph built from the same canonical vocabulary). The food-safety experiment in Section 6.9 partially relaxes this by evaluating on an *entirely unseen legal domain* with disjoint statutes and provisions, and the +6.5-point margin over the majority baseline there shows that at least some of the learned signal survives a hard entity shift. What that experiment does *not* measure is the finer-grained inductive generalisation within a familiar domain — e.g. a calendar-year or court hold-out, in which most entities are known but the specific case neighbourhood is new. The per-year error analysis already suggests older cases are harder, which is a prior on how such a split would behave, and it remains the natural follow-on experiment.

Outcome labels are LLM-generated. The win/loss labels are produced by prompting a Mistral model on the OpenNyAI-produced summary of

each case. The labels are therefore noisy with respect to the ground-truth disposition, and the noise is not uniform across buckets: sexual-offences and financial-fraud matters often produce multi-part dispositions that do not map cleanly onto a binary win/loss axis. The fact that these are also the two worst-performing buckets is at least partially a label-noise effect rather than a modelling failure. A multi-label or ordinal target is produced by the same pipeline and is an obvious next experiment.

RPC/RLC filtering is a lower bound on leakage, not a guarantee. The leakage-safety story combines a deterministic filter (rhetorical-role labels, field names, leakage phrases) with a structural argument (shared-entity types are law, not identity). Neither is complete: an RR classifier can mis-label a decisive sentence, a leakage phrase list is never exhaustive, and a sufficiently expressive GNN could in principle reconstruct a decisive fragment from non-decisive neighbours. The `no_names` result is evidence against one specific leakage mode (party identity) but not against all leakage modes. A manual audit of high-confidence correct predictions – sampled across buckets and rhetorical-role density – is the next concrete step.

The ablation ordering is stable but the margins are small. The best configuration beats the baseline by +1.7 points of accuracy on the pooled dataset; most other variants are within ±1 point. The 5-fold error bars are of the same order, which means that while the *qualitative* findings (text dominates, identity is removable, motor-accidents is easiest) are robust, the *quantitative* ordering of the close ablations should be reported as a cluster of comparable configurations rather than a strict ranking.

The pooled optimum is not the universal optimum. The new per-dataset best-variant table in Chapter 6 shows that the strongest global configuration is not the best choice in every domain: family-matrimonial, motor-accidents, and sexual-offences each favour different ablations under accuracy or macro-F1. This means the pooled cross-bucket score should be read as a useful compromise over heterogeneous graph geometries, not as evidence that one architectural setting is intrinsically best for all legal subdomains.

8.3 Future Work

Several directions follow naturally from the findings and limitations above, in rough order of expected payoff.

1. Inductive and temporal splits. The transductive protocol is the easier regime; a calendar-year hold-out (train on pre-2020, test on post-2020) and a court hold-out (train on all but one High Court, test on that one) would measure inductive generalisation directly. The connectivity-score machinery in Chapter 5 already provides the right diagnostic for these splits: a well-generalising model should remain competitive on the hold-out distribution's low-connectivity tail.

2. Multi-label and ordinal outcome targets. The Mistral labelling pipeline produces a multi-label variant alongside the single-label `win`/`loss` target used in this thesis. Moving the classifier head to a multi-label sigmoid (or to an ordinal regression over the $[-1, +1]$ score) would (i) absorb the disposition ambiguity currently pushed into the binary target and (ii) enable disposition-specific error analysis (e.g. *remanded* cases separately from *dismissed* cases), which the present evaluation cannot distinguish.

3. Expert-in-the-loop validation of the case-level explanations. Chapter 7 already produces per-case explanations: the `Graph_Analyser` pipeline extracts, for any test case, the top statutes, provisions, precedents, and argument spans the frozen HGT relied on, together with the FAISS-retrieved similar training cases and an LLM-written narrative grounded in that evidence bundle. The natural next step is the validation loop this machinery enables: sample predictions across confidence and difficulty strata, hand the Phase 4 evidence bundle and the Phase 5 narrative to a legal expert, and collect structured judgements on (i) whether the surfaced statutes and provisions are legally defensible reasons for the predicted outcome, (ii) whether the retrieved analogues are *legitimate* analogues rather than surface lookalikes, and (iii) whether the confident-wrong neighbour pattern identified in Chapter 7 corresponds to a coherent class of distinguishing-fact omissions. That turns the explainer from an internal diagnostic into an auditable artefact and is the cleanest path from the present offline evaluation to a deployable review tool.

4. Scaling the bucket inventory. Five legal domains is a deliberate scope; the pipeline is bucket-agnostic by construction, and adding new domains (tax, labour, consumer) would expand the cross-domain spine and, through shared IPC/CrPC hubs, improve signal on the existing buckets as well. The main engineering cost is the per-bucket Mistral labelling pass; the rest of the pipeline scales without modification.

5. Stronger identity-leakage audits. The `no_names` ablation is a single, coarse leakage probe. Finer probes – a judge-hold-out split to measure judge-identity leakage directly; a court-hold-out split to measure court-identity leakage; and a text-similarity-based nearest-neighbour attack to detect cases whose neighbours in the graph also share rare token n-grams – would tighten the leakage-safety claim from "no evidence of identity shortcuts" to "identity shortcuts are quantitatively bounded".

6. Cross-domain fine-tuning and domain-adaptive heads. The food-safety experiment in Section 6.9 establishes the cross-bucket HGT as a usable zero-shot base model on unseen Indian legal domains, but also quantifies the $\sim$20-point in-domain vs. out-of-domain gap. Two concrete follow-ons map directly onto that finding: (i) light-weight fine-tuning — freezing the HGT trunk and re-training only the classifier head (and optionally a small adapter over the CASE node) on a modestly-sized target-domain labelled sample, to quantify how quickly the gap closes as a function of labelled food-safety examples; and (ii) a domain-conditioned head that reads a domain embedding alongside the post-GNN case embedding, so that one unified model can be evaluated across all training domains *and* a held-out new domain without per-domain retraining. Both are natural complements to the zero-shot baseline reported here.

7. Connectivity- and difficulty-aware modelling. Two empirical regularities now point to a targeted next step. First, the pooled best ablation is not the best ablation inside every bucket. Second, the hardest held-out cases are the ones with long factual narratives or dense statute neighbourhoods. A natural extension is therefore a model that allocates capacity conditionally – for example, using richer hierarchical text encoding for document-heavy cases, stronger graph propagation for high-connectivity cases, or a domain-adaptive head once the bucket geometry is known. That would turn the descriptive

error analysis of Chapter 6 into an explicit modelling strategy.

Taken together, these findings and directions support a narrow and defensible thesis: legal outcome prediction over Indian court judgments *is* improvable by a heterogeneous, cross-case graph representation; the improvement is modest but measurable; it survives an aggressive identity-leakage audit; it produces predictions that can be audited case-by-case with legally named evidence; and it transfers partially, without fine-tuning, to a legal domain the model has never seen.

More strongly, the combination of schema design, leakage-filtered text construction, locality constraints on identity nodes, post-hoc ablation evidence, per-case interpretability via the `Graph_Analyser` pipeline, and a zero-shot out-of-domain test on food-safety judgments shows that this framework does not merely claim to avoid identity shortcuts and produce a usable prediction number; it operationally resists leakage while remaining predictive, it exposes its own confident failure mode through representation-space neighbour analysis, and it carries a non-trivial legal-reasoning prior across domain boundaries. That combination — leakage-safe construction, auditable predictions, and measured cross-domain transfer — is the main methodological contribution of the thesis, not just a side condition on the experimental setup.

Chapter 9

Conclusion

This thesis studied Legal Judgment Prediction for Indian court cases as a structured prediction problem rather than a plain document-classification task. It argued that credible legal prediction requires more than strong accuracy numbers: it requires a representation that preserves legally meaningful structure while actively resisting leakage from decision-bearing text and identity-based shortcuts.

To address this, the thesis introduced an end-to-end pipeline that converts raw Indian judgments into leakage-audited, graph-ready case records through entity extraction, rhetorical-role segmentation, outcome labelling, multi-hearing consolidation, and reasoning-focused graph construction. On top of this representation, a Heterogeneous Graph Transformer was used to model both the internal structure of each case and the shared legal links across cases through statutes, provisions, and precedents.

Across more than 73,000 Indian court cases spanning five legal domains, the proposed framework achieved strong pooled performance (0.8020 accuracy, 0.7959 macro-F1, 0.8831 ROC-AUC under 5-fold cross-validation on the best `party_args_lr_decay` configuration) and consistently outperformed text-only reference settings. More importantly, the results and ablations suggest that the gains are not coming merely from surface-level text patterns, but from cross-case legal structure encoded in a controlled and auditable way: a probing classifier isolates a +3.97-point accuracy gain attributable to graph message passing at fixed classifier capacity.

Beyond pooled accuracy, the thesis addressed the two questions a legal audience actually cares about: *can individual predictions be read*, and *does the model transfer beyond the distribution it was trained on*? A four-level

interpretability pipeline — representation probing, component-level importance, error decomposition, and the case-level `Graph_Analyser` built around a heterogeneous PGExplainer, FAISS retrieval in post-GNN space, and LLM verbalisation — produces grounded, legally named explanations for individual predictions and exposes a clean structural signature of the model's confident failures: when the prediction is confidently wrong, the retrieved representation-space neighbours almost always reinforce the *wrong* label. A zero-shot evaluation on 11,769 food-safety judgments, a legal domain absent from training, reaches 0.608 accuracy and 0.675 ROC-AUC against a 0.543 majority baseline without any fine-tuning, showing that the cross-bucket HGT learns a partially domain-general legal-reasoning prior rather than memorising training domains' statutory hubs.

The main contribution of this thesis is therefore methodological. It shows that legal outcome prediction over Indian judgments becomes more credible when the task is built around leakage-safe data construction, heterogeneous graph design, explicit constraints on what information may be shared across cases, per-case explainability tied to the same representation the model predicts from, and an out-of-domain evaluation that tests whether the learned signal is truly about legal reasoning or about corpus memorisation. While the improvements are measured rather than dramatic, they support a clear conclusion: graph-based legal prediction is most useful when it is not only accurate, but also structurally faithful, auditable at the case level, and experimentally honest about its transfer behaviour.

Future work can build on this foundation by tightening leakage audits further, extending the outcome space beyond binary decisions, designing models that adapt to domain-specific difficulty and long factual narratives, closing the in-domain/out-of-domain gap through light-weight fine-tuning or domain-adaptive heads, and pairing the case-level explainer with a legal-expert validation loop to convert it from an internal diagnostic into an auditable review tool.

Bibliography

[1] Nikolaos Aletras, Dimitrios Tsarapatsanis, Daniel Preǫiuc-Pietro, and Vasileios Lampos. Predicting judicial decisions of the European Court of Human Rights: a Natural Language Processing perspective. *PeerJ Computer Science*, 2:e93, 2016.

[2] Paheli Bhattacharya, Kaustubh Hiware, Subham Rajgaria, Nilay Pochhi, Kripabandhu Ghosh, and Saptarshi Ghosh. A comparative study of summarization algorithms applied to legal case judgments. In *Advances in Information Retrieval – ECIR 2019*, volume 11438 of *Lecture Notes in Computer Science*, pages 413–428. Springer, 2019.

[3] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904. Association for Computational Linguistics, 2020.

[4] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330. Association for Computational Linguistics, 2022.

[5] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pages 2704–2710. ACM, 2020.

[6] Abhinav Joshi, Shounak Paul, Avinash Anand, Md. Shad Akhtar, and Ashutosh Modi. IL-TUR: Benchmark for Indian legal text understanding

and reasoning. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 4035–4049. ELRA and ICCL, 2024.

[7] Prathamesh Kalamkar, Aman Tiwari, Astha Agarwal, Saurabh Karn, Smita Gupta, Vivek Raghavan, and Ashutosh Modi. Corpus for automatic structuring of legal documents. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 4420–4429. European Language Resources Association, 2022.

[8] Daniel Martin Katz, Michael James Bommarito, and Josh Blackman. A general approach for predicting the behavior of the Supreme Court of the United States. *PLOS ONE*, 12(4):e0174698, 2017.

[9] Shubham Khatri, Pooja Rani, Aaryan Mattoo, Bhanu Pratap Singh Rawat, Kshitij Gulati, Shouvik Sachdeva, Gaurav Gambhir, and Vikram Goyal. Legal judgment prediction with Indian court cases using graph neural networks. In *Proceedings of the 19th International Conference on Artificial Intelligence and Law*, pages 191–200. ACM, 2023.

[10] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[11] Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripa Ghosh, Shouvik Sengupta, Arnab Bhattacharya, and Ashutosh Modi. ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4046–4062. Association for Computational Linguistics, 2021.

[12] Dimitris Mamakas, Petros Tsotsi, Ion Androutsopoulos, and Ilias Chalkidis. Processing long legal documents with pre-trained transformers: Modding LegalBERT and Longformer. In *Proceedings of the Natural Legal Language Processing Workshop 2022*, pages 130–142. Association for Computational Linguistics, 2022.

[13] Masha Medvedeva, Michel Vols, and Martijn Wieling. Using machine learning to predict decisions of the European Court of Human Rights. *Artificial Intelligence and Law*, 28(2):237–266, 2020.

[14] Masha Medvedeva, Martijn Wieling, and Michel Vols. Rethinking the field of automatic prediction of court decisions. *Artificial Intelligence and Law*, 31(1):195–212, 2023.

[15] Shubham Nigam, Parth Mehta, Arnab Bhattacharya, and Prasenjit Majumder. NyayaAnumana & IndianLegalBench: Towards comprehensive Indian legal judgment prediction and reasoning. *arXiv preprint arXiv:2501.03745*, 2025.

[16] Shubham Kumar Nigam, Noel Shallum, Kripabandhu Ghosh, and Arnab Bhattacharya. LegalSeg: Unlocking the structure of Indian legal judgments through rhetorical role labeling. *arXiv preprint arXiv:2501.09690*, 2025.

[17] OpenNyAI. About OpenNyAI. `https://opennyai.org/about`, 2024. Accessed 2024.

[18] OpenNyAI. OpenNyAI documentation. `https://opennyai.org`, 2024. Accessed 2024.

[19] T.Y.S.S. Santosh, Vatsal Venkatkrishna, Saksham Argwal, and Matthias Grabmair. Leveraging precedents for legal judgment prediction in low-resource settings: A case study on the European Court of Human Rights. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 3887–3897. ELRA and ICCL, 2024.

[20] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web – ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607. Springer, 2018.

[21] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Peng Cui, Philip S. Yu, and Yanfang Ye. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032. ACM, 2019.

[22] Haotian Wu, Lingwei Wei, Mingze Li, Wei Zhou, Yan Zhao, Luqing Hou, Dapeng Liu, and Jine Tang. Precedent-enhanced legal judgment prediction with LLM and domain-model collaboration. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12060–12075. Association for Computational Linguistics, 2023.

[23] Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. Legal judgment prediction via topological learning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3540–3549. Association for Computational Linguistics, 2018.